المدرسة الوطنية للعلوم التطبيقية – أكادير
ⵜⵉⵎⵙⵙⵓⵜ ⵏ ⵜⵓⵙⵙⵓⵔⵉⵏ ⵜⵉⵖⵉⵙⵓⵖⵉⵏ - ⴰⴳⴰⴷⵉⵔ
ÉCOLE NATIONALE DES SCIENCES APPLIQUÉES - AGADIR

Rapport de projet académique

Présenté par :

Reda ERRAOUI

Spécialité : Finance et Ingénierie Décisionnelle

VALORISATION DES OPTIONS EUROPÉENNES PAR BLACK-SCHOLES, MONTE-CARLO, EULER-MARUYAMA ET MACHINE LEARNING

Encadré par : Pr. B. El Asri

Année universitaire : 2025–2026

Résumé

Ce rapport étudie la valorisation des options européennes à travers quatre approches complémentaires : le modèle analytique de Black–Scholes, la simulation de Monte–Carlo, le schéma d'Euler–Maruyama et une approche de Machine Learning. L'objectif est de comprendre comment le prix d'une option peut être obtenu à partir du principe d'absence d'arbitrage, puis approché numériquement lorsque les formules fermées deviennent insuffisantes.

Le modèle de Black–Scholes constitue la référence théorique du travail. Sous la mesure risque-neutre, le prix d'une option européenne s'exprime comme l'espérance actualisée de son payoff terminal. Cette formulation permet d'obtenir une formule fermée pour les options Call et Put, qui sert ensuite de base de comparaison pour les autres méthodes.

La méthode de Monte–Carlo approxime cette espérance par la moyenne empirique d'un grand nombre de scénarios simulés. Sa convergence est justifiée par la loi des grands nombres et le théorème central limite. Le schéma d'Euler–Maruyama permet, quant à lui, de discrétiser l'équation différentielle stochastique du sous-jacent afin de simuler des trajectoires complètes et d'analyser l'effet du pas de temps sur le prix obtenu.

Enfin, le Machine Learning est utilisé sous forme de *surrogate model*. Dans ce projet, un modèle Random Forest est entraîné à approximer la fonction de pricing à partir de prix générés par Black–Scholes. Il ne remplace donc pas la théorie financière, mais permet d'obtenir une prédiction rapide après apprentissage.

Ces méthodes sont intégrées dans une application web interactive développée en Python avec Flask, intitulée *Option Pricing Lab*. L'application permet de calculer les prix, de comparer les méthodes, de visualiser les trajectoires simulées, d'étudier la convergence numérique et d'exporter les résultats.

Mots clés : options européennes, Black–Scholes, Monte–Carlo, Euler–Maruyama, équations différentielles stochastiques, mesure risque-neutre, Machine Learning, Random Forest, surrogate model, pricing, Flask.

Liste des abréviations

ATM	At The Money (option à la monnaie, lorsque $S_0 \approx K$)
BS	Black–Scholes (modèle analytique de valorisation des options européennes)
BSM	Black–Scholes–Merton
EDS	Équation Différentielle Stochastique
EM	Euler–Maruyama (schéma de discrétisation d'une EDS)
GBM	Geometric Brownian Motion (Mouvement Brownien Géométrique)
MBG	Mouvement Brownien Géométrique
IC	Intervalle de Confiance
IID	Independent and Identically Distributed (variables indépendantes et identiquement distribuées)
ITM	In The Money (option dans la monnaie)
LHS	Latin Hypercube Sampling (échantillonnage par hypercube latin)
MAE	Mean Absolute Error (erreur absolue moyenne)
MC	Monte–Carlo
ML	Machine Learning (apprentissage automatique)
MLP	Multilayer Perceptron (réseau de neurones multicouche)
OTM	Out of The Money (option hors de la monnaie)
RF	Random Forest (forêt aléatoire)
RMSE	Root Mean Squared Error (racine de l'erreur quadratique moyenne)
RN	Risk-Neutral (neutre au risque)
RNG	Random Number Generator (générateur de nombres aléatoires)
SDE	Stochastic Differential Equation (équation différentielle stochastique)
TCL	Théorème Central Limite

Liste des figures

3.1 Principe du Random Forest Regressor : plusieurs arbres de décision sont entraînés sur des sous-échantillons bootstrap, puis leurs prédictions sont agrégées par moyenne afin d'obtenir le prix final de l'option. 32

4.1 Tableau de bord principal de l'application Option Pricing Lab. 44
4.2 Interface de calcul Black–Scholes. 45
4.3 Interface de simulation Monte–Carlo. 46
4.4 Courbe de convergence de la méthode Monte–Carlo. 46
4.5 Interface Euler–Maruyama. 47
4.6 Trajectoires simulées du sous-jacent par Euler–Maruyama. 48
4.7 Interface Machine Learning pour l'entraînement et la prédiction. 49
4.8 Interface de comparaison des méthodes de pricing. 50

Table des matières

Résumé **i**

Liste des abréviations **ii**

Introduction générale **1**

1 Cadre théorique du pricing des options européennes **3**

1.1 Introduction . . . 3

1.2 Introduction aux produits dérivés . . . 3

1.3 Définition d'une option européenne . . . 4

1.4 Notations principales . . . 4

1.4.1 Option Call européenne . . . 6

1.4.2 Option Put européenne . . . 6

1.5 Payoff des options européennes . . . 6

1.5.1 Payoff d'un Call européen . . . 6

1.5.2 Payoff d'un Put européen . . . 7

1.6 Moneyness d'une option . . . 7

1.7 Principe d'actualisation et cadre risque-neutre . . . 7

1.8 Parité Call–Put . . . 8

1.9 Le modèle de Black–Scholes et son extension avec dividende . . . 9

1.9.1 Hypothèses du modèle . . . 9

1.9.2 Dynamique du sous-jacent . . . 10

1.9.3 Origine de la solution explicite . . . 10

1.9.4 Formule du Call européen . . . 11

1.9.5 Formule du Put européen . . . 12

1.9.6 Interprétation de d_1 et d_2 . . . 12

1.9.7 Rôle des paramètres du modèle . . . 13

1.10 Mouvement brownien et équations différentielles stochastiques . . . 13

1.11 Limites du modèle de Black–Scholes–Merton . . . 14

1.12 Conclusion du chapitre . . . 15

2 Méthodes numériques de pricing : Monte–Carlo et Euler–Maruyama 16

2.1 Introduction . . . 16
2.2 Principe général de la méthode de Monte–Carlo . . . 16
2.3 Simulation des variables aléatoires . . . 17
2.4 Simulation directe du prix terminal de l'actif . . . 18
2.5 Application au modèle de Black–Scholes . . . 18
2.6 Pricing d'un Call européen par Monte–Carlo . . . 18
2.7 Pricing d'un Put européen par Monte–Carlo . . . 19
2.8 Erreur statistique et intervalle de confiance . . . 19
2.9 Algorithme de pricing Monte–Carlo . . . 20
2.10 Avantages et limites de la méthode de Monte–Carlo . . . 20
2.11 Principe de la discrétisation temporelle . . . 20
2.12 Schéma d'Euler–Maruyama . . . 21
2.12.1 Application au processus du sous-jacent . . . 21
2.13 Pricing des options européennes par Euler–Maruyama . . . 22
2.13.1 Estimation du prix d'un Call . . . 22
2.13.2 Estimation du prix d'un Put . . . 22
2.14 Algorithme Euler–Maruyama . . . 23
2.15 Biais de discrétisation et erreur numérique . . . 23
2.16 Influence du nombre de pas de temps . . . 24
2.17 Comparaison entre Monte–Carlo direct et Euler–Maruyama . . . 24
2.18 Conclusion du chapitre . . . 25

3 Machine Learning et comparaison numérique des méthodes de pricing 26

3.1 Introduction . . . 26
3.2 Introduction au Machine Learning en finance quantitative . . . 26
3.3 Machine Learning comme méthode d'approximation . . . 27
3.4 Construction de la base d'apprentissage . . . 28
3.4.1 Génération des paramètres financiers . . . 28
3.4.2 Calcul des prix cibles par Black–Scholes . . . 28
3.4.3 Échantillonnage des données . . . 29
3.5 Variables explicatives et variable cible . . . 29
3.5.1 Variables d'entrée . . . 29
3.5.2 Variables transformées . . . 29

3.5.3 Variable cible 30
3.6 Préparation des données 30
3.6.1 Séparation entre données d'entraînement et de test 30
3.6.2 Remarque sur la normalisation 31
3.7 Modèle de régression utilisé : Random Forest 31
3.8 Entraînement du modèle 32
3.9 Évaluation du modèle 33
3.9.1 Erreur absolue moyenne 33
3.9.2 Erreur quadratique moyenne 33
3.9.3 Coefficient de détermination 33
3.10 Prédiction du prix d'une option européenne 34
3.11 Intérêt et limites de l'approche Machine Learning 34
3.12 Objectif de la comparaison numérique 35
3.13 Paramètres numériques utilisés 35
3.14 Comparaison pour une option Call européenne 35
3.15 Comparaison pour une option Put européenne 36
3.16 Comparaison des prix obtenus 36
3.17 Comparaison des erreurs par rapport à Black–Scholes 36
3.18 Comparaison des temps de calcul 37
3.19 Analyse de la convergence Monte–Carlo 37
3.20 Analyse de l'effet du pas de discrétisation dans Euler–Maruyama 38
3.21 Analyse des performances du modèle Machine Learning 38
3.22 Synthèse comparative des méthodes 39
3.23 Conclusion du chapitre 39

4 Présentation de l'application développée 40
4.1 Introduction 40
4.2 Objectif de l'application 40
4.3 Technologies utilisées 41
4.4 Architecture générale de l'application 42
4.5 Structure générale du projet 43
4.6 Tableau de bord principal 44
4.7 Interface de calcul Black–Scholes 44
4.8 Interface de simulation Monte–Carlo 45

4.8.1 Graphe de convergence Monte–Carlo 46
4.9 Interface Euler–Maruyama 47
4.9.1 Trajectoires simulées 48
4.10 Interface Machine Learning 48
4.11 Tableau comparatif et visualisations graphiques 49
4.12 Export des résultats 51
4.13 Apports de l'application 51
4.14 Limites de l'application 51
4.15 Perspectives d'amélioration 52
4.16 Conclusion du chapitre 52

Conclusion générale **53**

Bibliographie **54**

Introduction générale

La finance moderne repose largement sur l'utilisation de modèles mathématiques permettant de représenter l'incertitude, d'évaluer les actifs financiers et d'analyser les risques liés aux décisions d'investissement. Parmi les instruments financiers les plus importants figurent les produits dérivés, dont la valeur dépend de l'évolution d'un actif sous-jacent. Les options financières occupent une place particulière dans cette famille, car elles donnent à leur détenteur un droit sans lui imposer une obligation. Elles sont ainsi utilisées aussi bien pour la couverture contre les risques que pour la spéculation ou la construction de stratégies financières plus complexes.

Une option européenne est un contrat donnant à son détenteur le droit, mais non l'obligation, d'acheter ou de vendre un actif sous-jacent à une date future donnée, appelée maturité, et à un prix fixé à l'avance, appelé prix d'exercice ou *strike*. Dans le cas d'une option d'achat, ou *Call*, le détenteur bénéficie d'une hausse du sous-jacent. Dans le cas d'une option de vente, ou *Put*, il bénéficie au contraire d'une baisse du sous-jacent. La question centrale du pricing est donc la suivante : quel est le prix juste aujourd'hui d'un droit portant sur une valeur future incertaine ?

Cette question est fondamentale en finance quantitative. En effet, une mauvaise valorisation peut conduire à des opportunités d'arbitrage, à une mauvaise couverture du risque ou encore à une interprétation incorrecte de la valeur réelle d'un instrument financier. Le principe d'absence d'arbitrage constitue alors le fondement théorique du pricing moderne. Il permet d'introduire la mesure risque-neutre, sous laquelle le prix d'une option européenne s'exprime comme l'espérance actualisée de son payoff terminal.

Le modèle de Black–Scholes constitue le point de départ classique de la valorisation des options européennes. Il repose sur l'hypothèse que le prix du sous-jacent suit un mouvement brownien géométrique, avec un taux sans risque et une volatilité constants. Sous ces hypothèses idéalisées, il est possible d'obtenir une formule analytique fermée pour le prix d'un Call ou d'un Put européen. Cette formule joue un rôle central dans ce rapport, car elle sert à la fois de référence théorique, de benchmark numérique et de générateur de prix cibles pour l'approche de Machine Learning.

Cependant, le cadre de Black–Scholes reste limité par ses hypothèses. Dans la réalité, la volatilité n'est pas toujours constante, les marchés peuvent connaître des sauts, les coûts de

transaction existent et certains produits financiers ne possèdent pas de formule analytique simple. C'est dans ce contexte que les méthodes numériques deviennent essentielles. La méthode de Monte–Carlo permet d'approximer une espérance par simulation aléatoire, tandis que le schéma d'Euler–Maruyama permet de discrétiser une équation différentielle stochastique afin de simuler des trajectoires complètes du sous-jacent.

Dans ce travail, une approche de Machine Learning est également étudiée sous la forme d'un *surrogate model*. L'objectif n'est pas de remplacer la théorie financière, mais d'apprendre une approximation rapide de la fonction de pricing. Plus précisément, un modèle Random Forest est entraîné à reproduire les prix générés par la formule de Black–Scholes à partir des paramètres financiers de l'option. Cette approche permet de montrer comment les méthodes statistiques peuvent compléter les modèles analytiques et numériques, notamment lorsque le calcul doit être répété un grand nombre de fois.

Ce rapport s'inscrit donc dans une logique progressive : partir du cadre théorique de Black–Scholes, comprendre le rôle de la simulation Monte–Carlo, étudier la discrétisation par Euler–Maruyama, puis introduire le Machine Learning comme outil d'approximation. L'objectif n'est pas seulement de présenter des formules, mais aussi d'expliquer leur origine, leur signification financière et leur utilisation dans une application concrète.

L'application développée, intitulée *Option Pricing Lab*, permet de mettre en pratique les différentes méthodes étudiées. Elle permet de calculer les prix des options européennes par Black–Scholes, Monte–Carlo, Euler–Maruyama et Random Forest, puis de comparer les résultats obtenus en termes de précision, d'erreur, de convergence et de temps de calcul. Elle offre également des visualisations graphiques, notamment les trajectoires simulées, la convergence Monte–Carlo et l'effet du pas de discrétisation dans Euler–Maruyama.

Le rapport est organisé en quatre chapitres. Le premier chapitre présente le cadre théorique du pricing des options européennes, en introduisant les produits dérivés, les payoffs, le principe d'actualisation, la mesure risque-neutre, le modèle de Black–Scholes et les équations différentielles stochastiques. Le deuxième chapitre est consacré aux méthodes numériques de pricing, notamment Monte–Carlo et Euler–Maruyama. Le troisième chapitre présente l'approche Machine Learning par Random Forest ainsi que la comparaison numérique des différentes méthodes. Enfin, le quatrième chapitre décrit l'application développée, ses interfaces, ses fonctionnalités de visualisation et ses possibilités d'export.

1 Cadre théorique du pricing des options européennes

1.1 Introduction

La valorisation des options financières constitue l'un des domaines fondamentaux de la finance quantitative. Elle repose sur l'utilisation de modèles mathématiques permettant de représenter l'évolution incertaine des marchés financiers et de déterminer le prix théorique d'un produit dérivé. Contrairement à un actif classique, dont la valeur est directement observée sur le marché, une option tire sa valeur d'un actif sous-jacent, tel qu'une action, un indice boursier, une devise ou une matière première.

L'objectif de ce chapitre est de présenter le cadre théorique nécessaire à la compréhension du pricing des options européennes. Nous introduisons d'abord les produits dérivés et les options européennes, puis nous présentons les notions de payoff, de moneyness et de parité Call–Put. Ensuite, nous détaillons le principe d'absence d'arbitrage et la mesure risque-neutre, qui constituent le fondement du pricing moderne. Enfin, nous étudions le modèle de Black–Scholes, ses hypothèses, la dynamique du sous-jacent, l'origine de la formule fermée et ses principales limites.

1.2 Introduction aux produits dérivés

Un produit dérivé est un instrument financier dont la valeur dépend de l'évolution d'un autre actif appelé actif sous-jacent. Ce sous-jacent peut être une action, un indice boursier, un taux d'intérêt, un taux de change, une matière première ou encore un portefeuille d'actifs. Les produits dérivés sont utilisés principalement pour trois objectifs : la couverture contre les risques, la spéculation et l'arbitrage.

La couverture consiste à utiliser un produit dérivé pour réduire l'exposition à un risque financier. Par exemple, un investisseur possédant une action peut acheter une option Put afin de se protéger contre une baisse future du prix de cette action. La spéculation consiste au contraire à prendre position sur l'évolution future du marché dans l'objectif de réaliser un gain.

Enfin, l'arbitrage consiste à exploiter des incohérences de prix entre plusieurs instruments financiers afin de réaliser un profit sans risque.

Parmi les produits dérivés, les options occupent une place centrale. Une option est un contrat donnant à son détenteur le droit, mais non l'obligation, d'acheter ou de vendre un actif sous-jacent à un prix fixé à l'avance, appelé prix d'exercice ou *strike*. Cette asymétrie est fondamentale : le détenteur exerce son droit uniquement si cela lui est favorable. Dans le cas contraire, il laisse l'option expirer sans l'exercer.

1.3 Définition d'une option européenne

Une option européenne est une option qui ne peut être exercée qu'à une date future déterminée, appelée maturité ou échéance. Elle se distingue d'une option américaine, qui peut être exercée à tout moment avant ou à la date d'échéance.

Une option européenne est caractérisée par les paramètres suivants :

— S_0 : le prix initial de l'actif sous-jacent ;
— K : le prix d'exercice, ou *strike* ;
— T : la maturité de l'option, exprimée en années ;
— r : le taux d'intérêt sans risque ;
— σ : la volatilité du sous-jacent ;
— le type de l'option : Call ou Put.

Le prix d'une option dépend donc à la fois de la valeur actuelle du sous-jacent, des conditions du marché et de l'incertitude future représentée par la volatilité.

1.4 Notations principales

Afin de clarifier les formules utilisées dans la suite du rapport, le tableau suivant regroupe les principales notations du chapitre.

Notation	**Signification**
S_0	Prix initial de l'actif sous-jacent à la date 0.
S_t	Prix du sous-jacent à une date intermédiaire t.
S_T	Prix du sous-jacent à la maturité T.
K	Prix d'exercice de l'option.
T	Maturité de l'option, exprimée en années.
r	Taux d'intérêt sans risque, supposé constant dans le modèle de Black–Scholes.
σ	Volatilité du sous-jacent, c'est-à-dire l'intensité des fluctuations aléatoires.
W_t	Mouvement brownien standard. Il modélise la source d'aléa du marché.
$\mathbb{P}$	Probabilité réelle, qui décrit la dynamique observée ou historique du marché.
$\mathbb{Q}$	Probabilité risque-neutre, utilisée pour valoriser les actifs dérivés sans arbitrage.
$\mathcal{F}_t$	Information disponible sur le marché jusqu'à la date t.
$\Pi(S_T)$	Payoff de l'option à la maturité.
V_0	Prix de l'option à la date initiale.
$N(\cdot)$	Fonction de répartition de la loi normale centrée réduite.
q	Taux de dividende continu du sous-jacent. Dans le cas classique sans dividende, $q = 0$.
d_1, d_2	Variables intermédiaires de la formule de Black–Scholes–Merton.

Table 1.1 – Principales notations utilisées dans le cadre du pricing des options européennes.

Cette table est importante car le pricing d'options repose sur plusieurs objets mathématiques : un prix aléatoire S_T, une fonction de payoff Π, une probabilité de valorisation $\mathbb{Q}$ et un facteur d'actualisation e^{-rT}.

1.4.1 Option Call européenne

Une option Call européenne donne à son détenteur le droit d'acheter l'actif sous-jacent au prix K à la date T. Si, à l'échéance, le prix du sous-jacent S_T est supérieur au strike K, le détenteur exerce l'option et achète l'actif à un prix inférieur à sa valeur de marché. Son gain est alors égal à $S_T - K$.

Si $S_T \leq K$, il n'a aucun intérêt à exercer l'option, car il pourrait acheter l'actif directement sur le marché à un prix inférieur ou égal à K. Dans ce cas, le payoff du Call est nul.

Le payoff d'un Call européen est donc :

$$\Pi_C(S_T) = \max(S_T - K, 0) = (S_T - K)^+.$$

1.4.2 Option Put européenne

Une option Put européenne donne à son détenteur le droit de vendre l'actif sous-jacent au prix K à la date T. Si, à l'échéance, le prix du sous-jacent S_T est inférieur au strike K, le détenteur exerce l'option et vend l'actif plus cher que sa valeur de marché. Son gain est alors égal à $K - S_T$.

Si $S_T \geq K$, l'option n'est pas exercée et le payoff est nul. Le payoff d'un Put européen est donc :

$$\Pi_P(S_T) = \max(K - S_T, 0) = (K - S_T)^+.$$

1.5 Payoff des options européennes

Le payoff représente le gain reçu par le détenteur de l'option à la maturité. Il ne doit pas être confondu avec le profit. Le payoff correspond au flux reçu à l'échéance, tandis que le profit tient compte de la prime payée initialement pour acheter l'option.

1.5.1 Payoff d'un Call européen

Le payoff d'un Call est une fonction croissante et convexe du prix terminal S_T. Lorsque S_T est inférieur ou égal à K, le payoff est nul. Lorsque S_T dépasse K, le payoff augmente linéairement.

$$\Pi_C(S_T) = \begin{cases} 0, & \text{si } S_T \leq K, \\ S_T - K, & \text{si } S_T > K. \end{cases}$$

Cette structure rend le Call attractif pour un investisseur anticipant une hausse du sous-jacent. La perte maximale est limitée à la prime payée, tandis que le gain potentiel est théoriquement illimité.

1.5.2 Payoff d'un Put européen

Le payoff d'un Put est une fonction décroissante et convexe du prix terminal S_T. Lorsque S_T est supérieur ou égal à K, le payoff est nul. Lorsque S_T devient inférieur à K, le payoff augmente à mesure que le sous-jacent baisse.

$$\Pi_P(S_T) = \begin{cases} K - S_T, & \text{si } S_T < K, \\ 0, & \text{si } S_T \geq K. \end{cases}$$

Le Put est souvent utilisé comme instrument de couverture, car il permet de protéger un portefeuille contre une baisse du marché.

1.6 Moneyness d'une option

La *moneyness* décrit la position du prix du sous-jacent par rapport au prix d'exercice. Elle permet de savoir si l'option possède déjà une valeur intrinsèque positive ou non.

On distingue trois situations :

— **In The Money (ITM)** : l'option possède une valeur intrinsèque positive ;

— **At The Money (ATM)** : le prix du sous-jacent est proche du strike ;

— **Out of The Money (OTM)** : l'option n'a pas de valeur intrinsèque.

Pour un Call, l'option est ITM lorsque $S_0 > K$, ATM lorsque $S_0 \approx K$, et OTM lorsque $S_0 < K$. Pour un Put, la logique est inversée : le Put est ITM lorsque $S_0 < K$.

La moneyness est une notion importante car elle influence directement le payoff potentiel, la valeur intrinsèque et la valeur temps de l'option.

1.7 Principe d'actualisation et cadre risque-neutre

Le pricing des options repose sur le principe d'absence d'opportunité d'arbitrage. Dans un marché sans arbitrage, deux portefeuilles produisant les mêmes flux futurs doivent avoir le même prix aujourd'hui. Si ce n'était pas le cas, un investisseur pourrait acheter le portefeuille le moins cher, vendre le plus cher et réaliser un profit sans risque.

Cette idée conduit au cadre risque-neutre. Sous la mesure risque-neutre, notée $\mathbb{Q}$, le rendement moyen du sous-jacent n'est plus le rendement espéré réel du marché, mais le taux sans risque r. Cette mesure ne sert donc pas à prévoir directement l'évolution réelle du marché ; elle sert à valoriser les actifs dérivés de manière cohérente avec l'absence d'arbitrage.

Le prix d'une option européenne de payoff $\Pi(S_T)$ est alors donné par l'espérance actualisée de son payoff terminal :

$$V_0 = e^{-rT}\mathbb{E}^{\mathbb{Q}}\left[\Pi(S_T)\right].$$

Plus généralement, à une date intermédiaire t, le prix s'écrit :

$$V_t = \mathbb{E}^{\mathbb{Q}}\left[e^{-r(T-t)}\Pi(S_T) \mid \mathcal{F}_t\right],$$

où $\mathcal{F}_t$ représente l'information disponible sur le marché jusqu'à la date t.

Le facteur $e^{-r(T-t)}$ correspond à l'actualisation continue. Il transforme un flux futur reçu à la date T en une valeur équivalente à la date t. Cette formule est centrale dans le rapport, car elle montre que le prix d'une option européenne est une espérance actualisée calculée sous une probabilité adaptée au pricing.

1.8 Parité Call–Put

La parité Call–Put est une relation fondamentale entre le prix d'un Call et celui d'un Put européens ayant le même sous-jacent, le même prix d'exercice et la même maturité. En l'absence de dividendes, elle s'écrit :

$$C - P = S_0 - Ke^{-rT}.$$

Cette relation peut être comprise en comparant deux portefeuilles :

— portefeuille A : acheter un Call et placer Ke^{-rT} au taux sans risque ;

— portefeuille B : acheter un Put et acheter le sous-jacent.

À la maturité, les deux portefeuilles donnent le même résultat quel que soit le niveau de S_T. Par absence d'arbitrage, ils doivent donc avoir le même prix aujourd'hui. Cette relation permet de vérifier la cohérence des prix calculés.

Dans le cas d'un dividende continu au taux q, la relation devient :

$$C - P = S_0 e^{-qT} - K e^{-rT}.$$

Cette extension est cohérente avec la formule de Black–Scholes–Merton utilisée dans l'application, où le paramètre q représente un taux de dividende continu.

1.9 Le modèle de Black–Scholes et son extension avec dividende

Le modèle de Black–Scholes constitue le point de départ classique de la valorisation des options européennes. Dans sa version de base, il suppose que le sous-jacent ne verse pas de dividendes. L'application développée utilise également une forme étendue avec dividende continu q, souvent appelée formule de Black–Scholes–Merton. Lorsque $q = 0$, cette formule se réduit au modèle classique de Black–Scholes.

Ce modèle fournit une formule analytique fermée pour le prix d'un Call ou d'un Put européen sous un ensemble d'hypothèses simplificatrices.

1.9.1 Hypothèses du modèle

Le modèle repose sur les hypothèses suivantes :

1. le prix du sous-jacent suit un mouvement brownien géométrique ;
2. le taux sans risque r est constant et connu ;
3. la volatilité σ est constante sur la durée de vie de l'option ;
4. les marchés sont liquides, sans coûts de transaction et sans taxes ;
5. les ventes à découvert sont autorisées ;
6. l'option est européenne, donc exerçable uniquement à la maturité T ;
7. le trading est continu ;
8. il n'existe aucune opportunité d'arbitrage.

Dans la version de base, le sous-jacent ne verse pas de dividendes. L'application permet toutefois d'introduire un taux de dividende continu q, ce qui correspond à l'extension de Merton. Lorsque $q = 0$, on retrouve la formule classique de Black–Scholes.

1.9.2 Dynamique du sous-jacent

Dans le cas sans dividende, le prix du sous-jacent suit, sous la mesure risque-neutre $\mathbb{Q}$, l'équation différentielle stochastique :

$$dS_t = rS_t dt + \sigma S_t dW_t.$$

Cette équation comporte deux composantes :

— $rS_t dt$: la composante déterministe, appelée dérive, qui représente la croissance moyenne au taux sans risque ;

— $\sigma S_t dW_t$: la composante aléatoire, qui représente les fluctuations du sous-jacent.

Lorsque l'on introduit un taux de dividende continu q, la dérive risque-neutre devient $r - q$. L'EDS s'écrit alors :

$$dS_t = (r - q)S_t dt + \sigma S_t dW_t.$$

Cette forme est celle utilisée dans la formule de Black–Scholes–Merton affichée par l'application.

1.9.3 Origine de la solution explicite

Dans le cas sans dividende, la solution de l'EDS :

$$dS_t = rS_t dt + \sigma S_t dW_t$$

peut être obtenue à l'aide de la formule d'Itô. On pose :

$$X_t = \ln(S_t).$$

La fonction $f(s) = \ln(s)$ vérifie :

$$f'(s) = \frac{1}{s}, \qquad f''(s) = -\frac{1}{s^2}.$$

D'après la formule d'Itô :

$$dX_t = f'(S_t)dS_t + \frac{1}{2}f''(S_t)(dS_t)^2.$$

Comme :

$$(dS_t)^2 = \sigma^2 S_t^2 dt,$$

on obtient :

$$dX_t = \left(r - \frac{1}{2}\sigma^2\right) dt + \sigma dW_t.$$

En intégrant entre 0 et T, on obtient :

$$\ln(S_T) - \ln(S_0) = \left(r - \frac{1}{2}\sigma^2\right) T + \sigma W_T.$$

Comme $W_T \sim \mathcal{N}(0, T)$, on peut écrire $W_T = \sqrt{T}Z$, avec $Z \sim \mathcal{N}(0, 1)$. Ainsi :

$$S_T = S_0 \exp\left[\left(r - \frac{1}{2}\sigma^2\right) T + \sigma\sqrt{T}Z\right], \qquad Z \sim \mathcal{N}(0, 1).$$

Le terme $-\frac{1}{2}\sigma^2$ provient de la correction d'Itô. Il ne s'agit donc pas d'un ajustement arbitraire, mais d'un terme mathématique qui apparaît lors de la transformation logarithmique d'un processus stochastique.

Avec dividende continu, la solution devient :

$$S_T = S_0 \exp\left[\left(r - q - \frac{1}{2}\sigma^2\right) T + \sigma\sqrt{T}Z\right].$$

Cette expression est importante pour la suite du rapport, car elle est utilisée directement dans la méthode de Monte–Carlo pour simuler le prix terminal du sous-jacent.

1.9.4 Formule du Call européen

Le prix d'un Call européen est donné par l'espérance actualisée de son payoff :

$$C = e^{-rT}\mathbb{E}^{\mathbb{Q}}\left[(S_T - K)^+\right].$$

La formule fermée de Black–Scholes–Merton est :

$$C = S_0 e^{-qT} N(d_1) - K e^{-rT} N(d_2),$$

avec :

$$d_1 = \frac{\ln(S_0/K) + \left(r - q + \frac{1}{2}\sigma^2\right) T}{\sigma\sqrt{T}}, \qquad d_2 = d_1 - \sigma\sqrt{T}.$$

Lorsque $q = 0$, on retrouve la formule classique de Black–Scholes :

$$C = S_0 N(d_1) - K e^{-rT} N(d_2).$$

Cette formule peut être interprétée comme la différence entre deux termes :

— $S_0 e^{-qT} N(d_1)$: la valeur actualisée du sous-jacent reçu dans les scénarios favorables ;

— $K e^{-rT} N(d_2)$: la valeur actualisée du prix d'exercice payé lorsque l'option est exercée.

1.9.5 Formule du Put européen

Le prix d'un Put européen est donné par :

$$P = K e^{-rT} N(-d_2) - S_0 e^{-qT} N(-d_1).$$

Lorsque $q = 0$, cette formule devient :

$$P = K e^{-rT} N(-d_2) - S_0 N(-d_1).$$

Le Put représente une protection contre la baisse du sous-jacent. Plus la probabilité que le sous-jacent termine sous le strike est élevée, plus la valeur du Put augmente.

1.9.6 Interprétation de d_1 et d_2

Les quantités d_1 et d_2 regroupent les principaux paramètres du modèle : la position du sous-jacent par rapport au strike, le taux sans risque, le dividende, la volatilité et la maturité.

Le terme d_2 est lié à la probabilité risque-neutre que le Call soit exercé :

$$\mathbb{Q}(S_T > K) = N(d_2).$$

Ainsi, $N(d_2)$ peut être vu comme une probabilité d'exercice sous la mesure risque-neutre. Le terme $N(d_1)$ intervient dans l'espérance du sous-jacent dans les scénarios où l'option termine dans la monnaie. Il ne s'agit donc pas simplement d'une probabilité réelle, mais d'un terme issu du calcul de l'espérance d'une variable log-normale tronquée.

La formule du Call peut donc se lire de manière financière :

Prix du Call = valeur actuelle attendue de l'actif reçu − valeur actuelle du strike payé.

Cette interprétation permet de comprendre la signification économique de la formule au-delà de son expression mathématique.

1.9.7 Rôle des paramètres du modèle

Chaque paramètre influence le prix de l'option :

— une hausse de S_0 augmente le prix du Call et diminue le prix du Put ;

— une hausse de K diminue le prix du Call et augmente le prix du Put ;

— une hausse de σ augmente généralement le prix du Call et du Put ;

— une hausse de T augmente généralement la valeur temps de l'option ;

— une hausse de r augmente généralement le prix du Call et diminue celui du Put ;

— une hausse de q diminue généralement le prix du Call et augmente celui du Put.

La volatilité joue un rôle central. Comme le payoff d'une option est asymétrique et borné inférieurement par zéro, une hausse de l'incertitude augmente la valeur potentielle des scénarios favorables sans créer une perte équivalente pour le détenteur de l'option.

1.10 Mouvement brownien et équations différentielles stochastiques

Le mouvement brownien est le processus aléatoire fondamental utilisé dans le modèle de Black–Scholes–Merton. Un mouvement brownien standard $(W_t)_{t\geq 0}$ vérifie :

1. $W_0 = 0$;
2. les accroissements sont indépendants ;
3. $W_t - W_s \sim \mathcal{N}(0, t-s)$ pour $0 \leq s < t$;
4. les trajectoires sont continues presque sûrement.

La propriété :

$$W_t - W_s \sim \mathcal{N}(0, t-s)$$

est essentielle pour la simulation. Sur un petit intervalle de temps Δt, on a :

$$\Delta W_j = W_{t_{j+1}} - W_{t_j} \sim \mathcal{N}(0, \Delta t).$$

On peut donc écrire :

$$\Delta W_j = \sqrt{\Delta t} Z_j, \qquad Z_j \sim \mathcal{N}(0,1).$$

Cette relation explique l'apparition du terme $\sqrt{\Delta t} Z_j$ dans le schéma d'Euler–Maruyama étudié dans le chapitre suivant.

Une équation différentielle stochastique générale s'écrit :

$$dX_t = \mu(X_t)dt + \sigma(X_t)dW_t,$$

où μ représente le coefficient de dérive et σ le coefficient de diffusion.

Dans le cas du modèle de Black–Scholes–Merton sans dividende :

$$\mu(S_t) = rS_t, \qquad \sigma(S_t) = \sigma S_t.$$

Avec dividende continu :

$$\mu(S_t) = (r - q)S_t.$$

Cette structure permet de faire le lien entre le modèle théorique de Black–Scholes–Merton, la simulation Monte–Carlo du prix terminal et la discrétisation par Euler–Maruyama.

1.11 Limites du modèle de Black–Scholes–Merton

Le modèle de Black–Scholes–Merton est une référence théorique importante, mais il repose sur des hypothèses restrictives. Il suppose notamment une volatilité constante, un taux sans risque constant, un marché liquide sans coûts de transaction et une dynamique continue du sous-jacent.

Ces hypothèses permettent d'obtenir une formule fermée, mais elles simplifient la réalité des marchés financiers. C'est pourquoi le modèle est utilisé dans ce rapport comme point de départ et comme référence de comparaison, et non comme description parfaite du marché réel.

Ces limites justifient l'utilisation de méthodes complémentaires. La méthode de Monte–Carlo permet d'estimer l'espérance actualisée du payoff par simulation. Le schéma d'Euler–Maruyama permet de simuler des trajectoires discrètes d'une équation différentielle stochastique. Enfin, le Machine Learning permet d'apprendre une approximation rapide de la fonction de pricing.

1.12 Conclusion du chapitre

Ce chapitre a présenté le cadre théorique du pricing des options européennes. Nous avons d'abord défini les options Call et Put, leurs payoffs et la notion de moneyness. Nous avons ensuite présenté le principe d'absence d'arbitrage, l'actualisation et la mesure risque-neutre, qui permettent d'écrire le prix d'une option comme l'espérance actualisée de son payoff terminal.

Le modèle de Black–Scholes, dans sa forme classique ou dans son extension avec dividende continu, fournit une formule analytique fermée pour les options européennes.

Ce cadre théorique constitue la base des chapitres suivants. La solution explicite du modèle sera utilisée dans la méthode de Monte–Carlo, tandis que l'EDS du sous-jacent servira de point de départ au schéma d'Euler–Maruyama. Enfin, les prix obtenus par Black–Scholes–Merton serviront de référence pour l'approche Machine Learning.

2 Méthodes numériques de pricing : Monte–Carlo et Euler–Maruyama

2.1 Introduction

Le modèle de Black–Scholes fournit une formule analytique exacte pour les options européennes standards. Cependant, dans de nombreux cas pratiques, les formules fermées ne sont pas disponibles. Les options peuvent dépendre de plusieurs facteurs de risque, de toute la trajectoire du sous-jacent, ou être définies dans des modèles plus complexes que le mouvement brownien géométrique. Dans ces situations, les méthodes numériques deviennent indispensables.

Ce chapitre présente deux méthodes numériques importantes : la méthode de Monte–Carlo et le schéma d'Euler–Maruyama. Monte–Carlo permet d'approcher une espérance par simulation, tandis qu'Euler–Maruyama permet de discrétiser une équation différentielle stochastique afin de simuler des trajectoires du sous-jacent.

2.2 Principe général de la méthode de Monte–Carlo

La méthode de Monte–Carlo est une méthode numérique basée sur la simulation aléatoire. Elle permet d'approximer une espérance mathématique par une moyenne empirique.

Soit X une variable aléatoire et g une fonction mesurable telle que $\mathbb{E}[|g(X)|] < \infty$. L'objectif est d'estimer :

$$\mu = \mathbb{E}[g(X)].$$

Si $X_1, \ldots, X_M$ sont des réalisations indépendantes de X, alors :

$$\widehat{\mu}_M = \frac{1}{M} \sum_{i=1}^{M} g(X_i).$$

D'après la loi forte des grands nombres :

$$\widehat{\mu}_M \xrightarrow[M\to\infty]{p.s.} \mu.$$

La loi forte des grands nombres justifie donc la convergence de l'estimateur. Cependant, elle ne donne pas directement la vitesse de convergence. Pour mesurer cette vitesse, on utilise le théorème central limite. Si la variance de $g(X)$ est finie, alors :

$$\sqrt{M}\,(\widehat{\mu}_M - \mu) \Longrightarrow \mathcal{N}(0, \sigma_g^2),$$

où $\sigma_g^2 = \mathrm{Var}(g(X))$. Cette relation montre que l'erreur de Monte–Carlo décroît en ordre :

$$O\left(\frac{1}{\sqrt{M}}\right).$$

Autrement dit, la précision augmente lentement : pour diviser l'erreur par deux, il faut multiplier le nombre de simulations par quatre.

En finance, le prix d'une option européenne est une espérance actualisée. La méthode de Monte–Carlo est donc naturellement adaptée au pricing des options.

2.3 Simulation des variables aléatoires

Toute simulation Monte–Carlo repose sur la génération de variables aléatoires. Dans le cadre du modèle de Black–Scholes, la variable aléatoire fondamentale est la loi normale centrée réduite. Elle intervient directement dans la solution explicite du mouvement brownien géométrique.

On note :

$$Z \sim \mathcal{N}(0, 1).$$

Dans l'application, les variables normales sont générées numériquement à l'aide des fonctions de génération pseudo-aléatoire de NumPy. Chaque réalisation de Z représente un scénario possible du choc aléatoire affectant le prix du sous-jacent.

Sous la mesure risque-neutre, la valeur terminale du sous-jacent s'écrit :

$$S_T = S_0 \exp\left[\left(r - \frac{1}{2}\sigma^2\right)T + \sigma\sqrt{T}Z\right].$$

Ainsi, pour obtenir plusieurs scénarios possibles, il suffit de générer plusieurs variables normales indépendantes :

$$Z_1, Z_2, \ldots, Z_M \sim \mathcal{N}(0, 1),$$

puis de calculer les prix terminaux correspondants :

$$S_T^{(i)} = S_0 \exp\left[\left(r - \frac{1}{2}\sigma^2\right)T + \sigma\sqrt{T}Z_i\right].$$

Cette étape constitue la base de la méthode Monte–Carlo directe utilisée pour les options européennes standards.

2.4 Simulation directe du prix terminal de l'actif

La méthode Monte–Carlo directe exploite la solution exacte du mouvement brownien géométrique. Pour chaque simulation i, on génère une variable normale Z_i, puis on calcule :

$$S_T^{(i)} = S_0 \exp\left[\left(r - \frac{1}{2}\sigma^2\right)T + \sigma\sqrt{T}Z_i\right].$$

Cette méthode est dite directe car elle ne simule pas toute la trajectoire du sous-jacent. Elle simule uniquement la valeur finale S_T, ce qui est suffisant pour les options européennes standards dont le payoff dépend seulement de la valeur du sous-jacent à maturité.

2.5 Application au modèle de Black–Scholes

Sous la mesure risque-neutre, le prix d'une option européenne est :

$$V_0 = e^{-rT}\mathbb{E}^{\mathbb{Q}}[\Pi(S_T)].$$

Monte–Carlo remplace l'espérance théorique par une moyenne empirique :

$$\widehat{V}_{MC} = e^{-rT}\frac{1}{M}\sum_{i=1}^{M}\Pi(S_T^{(i)}).$$

Cette formule est utilisée dans l'application pour calculer le prix simulé d'un Call ou d'un Put européen.

2.6 Pricing d'un Call européen par Monte–Carlo

Pour un Call européen, le payoff est :

$$\Pi_C(S_T) = \max(S_T - K, 0).$$

L'estimateur Monte–Carlo du prix du Call est donc :

$$\widehat{C}_{MC} = e^{-rT} \frac{1}{M} \sum_{i=1}^{M} \max(S_T^{(i)} - K, 0).$$

2.7 Pricing d'un Put européen par Monte–Carlo

Pour un Put européen, le payoff est :

$$\Pi_P(S_T) = \max(K - S_T, 0).$$

L'estimateur Monte–Carlo du prix du Put est :

$$\widehat{P}_{MC} = e^{-rT} \frac{1}{M} \sum_{i=1}^{M} \max(K - S_T^{(i)}, 0).$$

2.8 Erreur statistique et intervalle de confiance

L'estimateur Monte–Carlo est aléatoire. Deux exécutions avec des graines différentes peuvent donner des valeurs légèrement différentes. Il est donc nécessaire de mesurer l'incertitude statistique de l'estimation.

L'erreur standard est donnée par :

$$SE = e^{-rT} \frac{\widehat{\sigma}_{\Pi}}{\sqrt{M}},$$

où $\widehat{\sigma}_{\Pi}$ est l'écart-type empirique des payoffs.

Le coefficient 1.96 provient de la loi normale centrée réduite : environ 95% de la masse d'une loi normale se trouve entre -1.96 et 1.96.

Un intervalle de confiance à 95% est donné par :

$$IC_{95\%} = \left[\widehat{V}_{MC} - 1.96SE, \widehat{V}_{MC} + 1.96SE\right].$$

L'erreur Monte–Carlo décroît en :

$$O\left(\frac{1}{\sqrt{M}}\right).$$

Cela signifie que pour diviser l'erreur par deux, il faut multiplier le nombre de simulations par quatre.

Dans l'application, cette propriété est illustrée par une courbe de convergence. Lorsque le nombre de simulations M augmente, le prix estimé par Monte–Carlo se stabilise

progressivement autour du prix de référence donné par Black–Scholes. L'intervalle de confiance se resserre également, ce qui traduit une diminution de l'incertitude statistique.

2.9 Algorithme de pricing Monte–Carlo

Algorithme 1 : Pricing Monte–Carlo d'une option européenne

Input : S_0, K, r, σ, T, M, type de l'option

Output : Prix estimé $\widehat{V}_{MC}$

for $i \leftarrow 1$ **to** M **do**

Générer $Z_i \sim \mathcal{N}(0,1)$;

Calculer $S_T^{(i)} = S_0 \exp\left[\left(r - \frac{1}{2}\sigma^2\right)T + \sigma\sqrt{T}Z_i\right]$;

Calculer le payoff $\Pi(S_T^{(i)})$;

Calculer $\widehat{V}_{MC} = e^{-rT}\frac{1}{M}\sum_{i=1}^{M}\Pi(S_T^{(i)})$;

return $\widehat{V}_{MC}$

2.10 Avantages et limites de la méthode de Monte–Carlo

La méthode de Monte–Carlo présente plusieurs avantages :

— elle est simple à comprendre et à implémenter ;

— elle est flexible ;

— elle s'applique aux problèmes de grande dimension ;

— elle peut être parallélisée facilement ;

— elle permet de traiter des payoffs complexes.

Cependant, elle présente aussi des limites :

— sa convergence est lente ;

— elle nécessite un grand nombre de simulations pour obtenir une forte précision ;

— le résultat dépend de la graine aléatoire ;

— elle peut être coûteuse en temps de calcul.

2.11 Principe de la discrétisation temporelle

Contrairement à Monte–Carlo direct, qui simule directement S_T, le schéma d'Euler–Maruyama simule toute la trajectoire du sous-jacent sur une grille temporelle.

On divise l'intervalle $[0, T]$ en N pas de temps :

$$\Delta t = \frac{T}{N}.$$

Les dates de la grille sont :

$$t_j = j\Delta t, \qquad j = 0, \dots, N.$$

L'objectif est d'approcher le processus continu (S_t) par une suite discrète :

$$S_{t_0}, S_{t_1}, \dots, S_{t_N}.$$

2.12 Schéma d'Euler–Maruyama

Pour une EDS générale :

$$dX_t = b(t, X_t)dt + a(t, X_t)dW_t,$$

le schéma d'Euler–Maruyama est :

$$X_{t_{j+1}} = X_{t_j} + b(t_j, X_{t_j})\Delta t + a(t_j, X_{t_j})\Delta W_j,$$

avec :

$$\Delta W_j = \sqrt{\Delta t} Z_j, \qquad Z_j \sim \mathcal{N}(0, 1).$$

Cette écriture vient de la propriété fondamentale du mouvement brownien :

$$W_{t_{j+1}} - W_{t_j} \sim \mathcal{N}(0, t_{j+1} - t_j).$$

Comme $t_{j+1} - t_j = \Delta t$, on a :

$$\Delta W_j \sim \mathcal{N}(0, \Delta t).$$

Or, si $Z_j \sim \mathcal{N}(0, 1)$, alors $\sqrt{\Delta t} Z_j \sim \mathcal{N}(0, \Delta t)$. C'est pourquoi le terme aléatoire du schéma d'Euler–Maruyama s'écrit avec $\sqrt{\Delta t} Z_j$.

2.12.1 Application au processus du sous-jacent

Dans le modèle de Black–Scholes :

$$dS_t = rS_t dt + \sigma S_t dW_t.$$

Le schéma d'Euler–Maruyama devient :

$$S_{t_{j+1}} = S_{t_j} + rS_{t_j}\Delta t + \sigma S_{t_j}\sqrt{\Delta t} Z_j.$$

On peut aussi écrire :

$$S_{t_{j+1}} = S_{t_j}\left(1 + r\Delta t + \sigma\sqrt{\Delta t} Z_j\right).$$

2.13 Pricing des options européennes par Euler–Maruyama

Après avoir simulé M trajectoires du sous-jacent, on récupère les valeurs finales $S_N^{(i)}$, puis on calcule les payoffs et on les actualise.

Le prix estimé par Euler–Maruyama est :

$$\widehat{V}_{EM} = e^{-rT}\frac{1}{M}\sum_{i=1}^{M}\Pi(S_N^{(i)}).$$

2.13.1 Estimation du prix d'un Call

$$\widehat{C}_{EM} = e^{-rT}\frac{1}{M}\sum_{i=1}^{M}\max(S_N^{(i)} - K, 0).$$

2.13.2 Estimation du prix d'un Put

$$\widehat{P}_{EM} = e^{-rT}\frac{1}{M}\sum_{i=1}^{M}\max(K - S_N^{(i)}, 0).$$

2.14 Algorithme Euler–Maruyama

Algorithme 2 : Pricing d'une option européenne par Euler–Maruyama

Input : S_0, K, r, σ, T, M, N, type de l'option

Output : Prix estimé $\widehat{V}_{EM}$

Calculer $\Delta t = T/N$;

for $i \leftarrow 1$ **to** M **do**

  Initialiser $S_0^{(i)} = S_0$;

  for $j \leftarrow 0$ **to** $N-1$ **do**

    Générer $Z_j^{(i)} \sim \mathcal{N}(0,1)$;

    $S_{j+1}^{(i)} = S_j^{(i)} + rS_j^{(i)}\Delta t + \sigma S_j^{(i)}\sqrt{\Delta t}Z_j^{(i)}$;

  Calculer le payoff terminal $\Pi(S_N^{(i)})$;

Calculer $\widehat{V}_{EM} = e^{-rT}\frac{1}{M}\sum_{i=1}^{M}\Pi(S_N^{(i)})$;

return $\widehat{V}_{EM}$

2.15 Biais de discrétisation et erreur numérique

Euler–Maruyama introduit un biais de discrétisation. Ce biais provient du fait que la trajectoire continue est approximée par une suite de valeurs discrètes. L'erreur dépend du pas de temps :

$$\Delta t = \frac{T}{N}.$$

Plus N est grand, plus Δt est petit, et plus l'approximation est précise. Pour une approximation d'espérance, on parle généralement d'erreur faible. Dans le cas du schéma d'Euler–Maruyama, cette erreur est souvent d'ordre :

$$O(\Delta t).$$

Comme $\Delta t = T/N$, cela revient à écrire, à maturité fixée :

$$O\left(\frac{1}{N}\right).$$

Il faut distinguer deux sources d'erreur :

— l'erreur statistique, liée au nombre de simulations M ;

— l'erreur de discrétisation, liée au nombre de pas N.

Augmenter M réduit l'erreur statistique, mais ne supprime pas le biais de discrétisation. Pour réduire ce biais, il faut augmenter N.

2.16 Influence du nombre de pas de temps

Le choix de N est essentiel. Si N est trop faible, la trajectoire est approximée de manière grossière. Si N est très grand, le résultat devient plus précis, mais le temps de calcul augmente.

La complexité d'Euler–Maruyama est :

$$O(MN).$$

Dans l'application, le choix $N = 252$ est naturel car il correspond approximativement au nombre de jours de bourse dans une année. Cette influence est visualisée à travers un graphique comparant le prix obtenu par Euler–Maruyama pour différentes valeurs de N. Lorsque N augmente, le pas Δt diminue et le biais par rapport au prix de référence Black–Scholes tend généralement à se réduire.

2.17 Comparaison entre Monte–Carlo direct et Euler–Maruyama

Critère	**Monte–Carlo direct**	**Euler–Maruyama**
Valeur simulée	Prix terminal S_T	Trajectoire complète
Solution utilisée	Solution exacte du MBG	Approximation discrète
Biais de discrétisation	Aucun, car S_T est simulé directement	Présent, d'ordre faible $O(1/N)$
Erreur statistique	$O(1/\sqrt{M})$	$O(1/\sqrt{M})$
Complexité	$O(M)$	$O(MN)$
Usage principal	Options européennes simples	Options dépendant du chemin

Table 2.1 – Comparaison entre Monte–Carlo direct et Euler–Maruyama.

2.18 Conclusion du chapitre

Ce chapitre a présenté les deux méthodes numériques principales du projet. La méthode de Monte–Carlo permet d'approximer l'espérance actualisée du payoff en simulant un grand nombre de valeurs terminales du sous-jacent. Sa justification repose sur la loi des grands nombres et sur le théorème central limite, qui expliquent la convergence de l'estimateur et la construction d'un intervalle de confiance.

Le schéma d'Euler–Maruyama permet, quant à lui, de discrétiser l'équation différentielle stochastique du sous-jacent afin de simuler des trajectoires complètes. Cette méthode introduit une erreur de discrétisation liée au pas de temps Δt, en plus de l'erreur statistique liée au nombre de simulations M.

La comparaison entre Monte–Carlo direct et Euler–Maruyama montre que les deux méthodes n'ont pas exactement le même rôle. Monte–Carlo direct est plus efficace pour les options européennes simples, car le payoff dépend uniquement de S_T. Euler–Maruyama est plus général, car il permet de construire toute la trajectoire du sous-jacent et devient utile lorsque le modèle ou le payoff nécessite une discrétisation temporelle.

3 Machine Learning et comparaison numérique des méthodes de pricing

3.1 Introduction

Après avoir étudié le modèle analytique de Black–Scholes ainsi que les méthodes numériques de Monte–Carlo et d'Euler–Maruyama, ce chapitre introduit une approche complémentaire fondée sur le Machine Learning. L'objectif n'est pas de remplacer les modèles financiers classiques, mais d'utiliser l'apprentissage automatique comme une méthode d'approximation rapide de la fonction de pricing.

Autrement dit, le Machine Learning intervient après la construction d'une référence théorique. Il ne crée pas une nouvelle formule financière : il apprend à reproduire une fonction de pricing déjà définie, ici celle de Black–Scholes. Cette distinction est importante, car elle permet de comprendre que la qualité du modèle dépend directement de la qualité des prix utilisés comme cibles.

Dans le cadre de ce projet, le Machine Learning est utilisé comme un *surrogate model*, c'est-à-dire un modèle approché capable d'apprendre la relation entre les paramètres d'une option européenne et son prix théorique. Les prix utilisés comme valeurs cibles sont générés par la formule de Black–Scholes. Ainsi, le modèle apprend à reproduire une fonction connue, ce qui permet d'évaluer précisément sa qualité de prédiction.

Ce chapitre présente d'abord le principe de l'approche Machine Learning appliquée au pricing des options. Il détaille ensuite la construction de la base d'apprentissage, les variables utilisées, le modèle de régression retenu, à savoir le Random Forest, ainsi que les métriques d'évaluation. Enfin, il propose une comparaison numérique globale entre Black–Scholes, Monte–Carlo, Euler–Maruyama et Machine Learning.

3.2 Introduction au Machine Learning en finance quantitative

Le Machine Learning désigne un ensemble de méthodes permettant à un modèle d'apprendre automatiquement des relations à partir de données. En finance quantitative, ces

méthodes sont utilisées pour plusieurs objectifs : prévision de séries financières, détection d'anomalies, gestion du risque, calibration de modèles, classification de profils de marché et approximation de fonctions complexes.

Dans le domaine du pricing d'options, le Machine Learning peut être utilisé pour approximer une fonction de valorisation. Lorsque le calcul exact ou numérique d'un prix est coûteux, il peut être intéressant d'entraîner un modèle sur un grand nombre d'exemples, puis d'utiliser ce modèle pour produire des prédictions rapides.

Cette approche est particulièrement utile lorsque :

— le calcul analytique n'est pas disponible ;
— la simulation numérique est coûteuse ;
— un très grand nombre d'options doit être valorisé rapidement ;
— l'on souhaite disposer d'un modèle de prédiction instantané après entraînement.

Dans notre cas, l'approche est utilisée dans un objectif pédagogique et comparatif. Le prix exact Black–Scholes sert de référence, et le modèle de Machine Learning apprend à approximer ce prix à partir des paramètres financiers.

3.3 Machine Learning comme méthode d'approximation

Le pricing d'une option européenne peut être vu comme une fonction reliant un vecteur de paramètres financiers à un prix :

$$f : (S_0, K, r, \sigma, T, \text{type}) \longrightarrow V.$$

où :

— S_0 est le prix initial du sous-jacent ;
— K est le prix d'exercice ;
— r est le taux sans risque ;
— σ est la volatilité ;
— T est la maturité ;
— *type* indique s'il s'agit d'un Call ou d'un Put ;
— V est le prix de l'option.

L'objectif du Machine Learning est de construire une approximation $\widehat{f}$ de la fonction de pricing :

$$\widehat{V} = \widehat{f}(S_0, K, r, \sigma, T, \text{type}).$$

Le modèle est entraîné à partir d'une base de données contenant plusieurs exemples :

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{n},$$

où x_i représente les paramètres de l'option et y_i le prix cible calculé par Black–Scholes.

3.4 Construction de la base d'apprentissage

3.4.1 Génération des paramètres financiers

La première étape consiste à générer un ensemble de paramètres financiers. Afin que le modèle apprenne correctement la relation entre les paramètres et le prix, il est nécessaire de couvrir un espace suffisamment large de situations.

Dans notre application, les paramètres peuvent être générés dans les intervalles suivants :

Paramètre	**Plage**	**Interprétation**
S_0	$[50, 200]$	Prix initial du sous-jacent.
K	$[40, 220]$	Prix d'exercice de l'option.
r	$[0, 0.10]$	Taux sans risque compris entre 0% et 10%.
σ	$[0.05, 0.80]$	Volatilité comprise entre 5% et 80%.
T	$[0.1, 3]$	Maturité comprise entre 0.1 et 3 ans.
Type	Call / Put	Type de l'option européenne.

Table 3.1 – Plages de génération des paramètres financiers pour la base d'apprentissage.

Ces plages permettent de représenter différentes situations : options à la monnaie, dans la monnaie, hors de la monnaie, maturités courtes ou longues, actifs peu volatils ou très volatils.

3.4.2 Calcul des prix cibles par Black–Scholes

Une fois les paramètres générés, le prix cible de chaque option est calculé à l'aide de la formule de Black–Scholes. Pour chaque observation x_i, on obtient donc :

$$y_i = V_{BS}(x_i).$$

Le choix de Black–Scholes comme générateur de prix cibles présente deux avantages. Premièrement, il permet d'obtenir des prix exacts dans le cadre du modèle. Deuxièmement, il permet d'évaluer objectivement les performances du modèle de Machine Learning, car l'erreur peut être mesurée directement par rapport à la référence analytique.

3.4.3 Échantillonnage des données

La génération des données peut être effectuée de manière aléatoire uniforme ou par une méthode plus structurée telle que le *Latin Hypercube Sampling*. Cette dernière permet de mieux couvrir l'espace des paramètres en évitant de concentrer trop d'observations dans certaines zones.

L'objectif est de construire une base suffisamment diversifiée pour que le modèle apprenne non seulement les cas standards, mais aussi les situations extrêmes : volatilité élevée, maturité courte, option fortement dans la monnaie ou fortement hors de la monnaie.

3.5 Variables explicatives et variable cible

3.5.1 Variables d'entrée

Les variables d'entrée du modèle sont les paramètres financiers de l'option :

$$S_0, \quad K, \quad r, \quad \sigma, \quad T, \quad \text{type}.$$

Le type de l'option est transformé en variable numérique. Par exemple, on peut utiliser :

$$\text{type} = 1 \quad \text{pour un Call},$$

$$\text{type} = 0 \quad \text{pour un Put}.$$

3.5.2 Variables transformées

En plus des variables brutes, il est utile d'ajouter des variables transformées qui reprennent la structure mathématique de la formule de Black–Scholes. Parmi ces variables, on peut citer :

$$\ln\left(\frac{S_0}{K}\right), \qquad \sqrt{T}, \qquad \sigma\sqrt{T}, \qquad e^{-rT}, \qquad \frac{S_0}{K}.$$

Ces transformations facilitent l'apprentissage, car elles donnent directement au modèle des informations importantes sur la moneyness, la volatilité totale et l'actualisation.

Variable	**Expression**	**Rôle financier**
Log-moneyness	$\ln(S_0/K)$	Position relative du sous-jacent par rapport au strike.
Racine de maturité	$\sqrt{T}$	Terme présent dans la volatilité totale.
Volatilité totale	$\sigma\sqrt{T}$	Mesure du risque total sur l'horizon T.
Facteur d'actualisation	e^{-rT}	Valeur actuelle d'un flux reçu à maturité.
Ratio sous-jacent/strike	S_0/K	Mesure simple de la moneyness.

Table 3.2 – Variables transformées utilisées pour enrichir la base d'apprentissage.

3.5.3 Variable cible

La variable cible est le prix théorique de l'option :

$$y = V_{BS}.$$

Le modèle cherche donc à minimiser l'écart entre le prix prédit $\widehat{y}$ et le prix cible y.

3.6 Préparation des données

3.6.1 Séparation entre données d'entraînement et de test

La base de données est divisée en deux parties :

— un ensemble d'entraînement, utilisé pour ajuster le modèle ;

— un ensemble de test, utilisé pour évaluer sa capacité de généralisation.

Une séparation classique consiste à utiliser 80% des données pour l'entraînement et 20% pour le test.

$$\mathcal{D} = \mathcal{D}_{train} \cup \mathcal{D}_{test}.$$

L'ensemble de test ne doit pas être utilisé pendant l'entraînement. Il permet de vérifier si le modèle est capable de prédire correctement des prix qu'il n'a jamais vus auparavant.

3.6.2 Remarque sur la normalisation

Dans le cas du Random Forest, la normalisation des variables n'est pas indispensable. En effet, les arbres de décision découpent l'espace des variables à partir de seuils successifs, et ils sont donc moins sensibles à l'échelle des données qu'un réseau de neurones.

Dans ce projet, l'accent est plutôt mis sur l'enrichissement des variables d'entrée à travers des transformations financières comme la log-moneyness, la volatilité totale $\sigma\sqrt{T}$ ou le facteur d'actualisation e^{-rT}. Ces variables transformées reprennent directement des éléments présents dans la formule de Black–Scholes et facilitent l'apprentissage de la fonction de pricing.

3.7 Modèle de régression utilisé : Random Forest

Le Random Forest Regressor est un modèle d'ensemble basé sur plusieurs arbres de décision. Chaque arbre est entraîné sur un sous-échantillon des données et produit une prédiction. La prédiction finale est obtenue en moyennant les prédictions de tous les arbres :

$$\widehat{y}_{RF} = \frac{1}{B}\sum_{b=1}^{B} \widehat{y}_b,$$

où B est le nombre d'arbres.

L'idée centrale du Random Forest est le *bagging*. Chaque arbre est entraîné sur un échantillon bootstrap, c'est-à-dire un échantillon obtenu par tirage avec remise à partir de la base d'apprentissage. Ainsi, les arbres ne voient pas exactement les mêmes données et produisent des erreurs différentes.

La moyenne des prédictions permet alors de réduire la variance du modèle. Un arbre de décision seul peut être instable : une petite modification des données peut changer fortement sa structure. En revanche, la moyenne de plusieurs arbres rend la prédiction plus stable, surtout lorsque les arbres ne sont pas parfaitement corrélés.

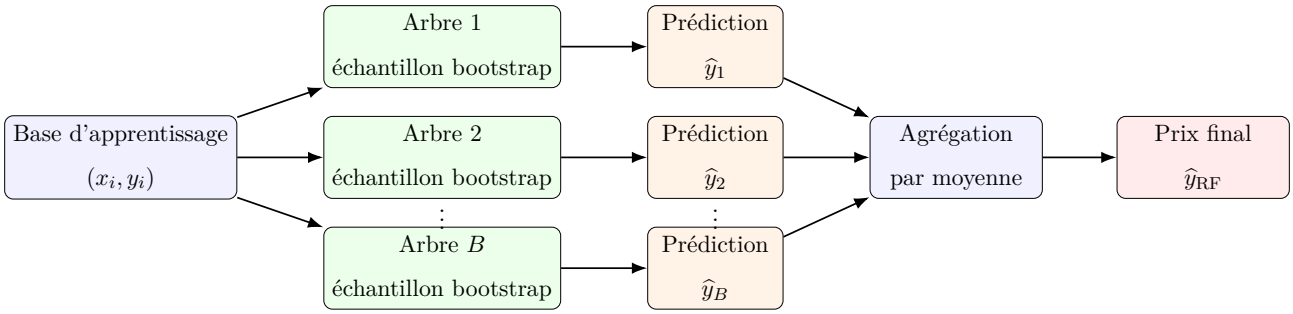


Figure 3.1 – Principe du Random Forest Regressor : plusieurs arbres de décision sont entraînés sur des sous-échantillons bootstrap, puis leurs prédictions sont agrégées par moyenne afin d'obtenir le prix final de l'option.

La figure 3.1 illustre le principe général du Random Forest Regressor. À partir de la base d'apprentissage, plusieurs arbres de décision sont construits sur des sous-échantillons bootstrap. Chaque arbre produit une estimation du prix de l'option, puis les prédictions individuelles sont moyennées afin d'obtenir la prédiction finale $\widehat{y}_{RF}$. Cette logique d'ensemble permet de réduire la variance du modèle, de limiter la sensibilité aux fluctuations de l'échantillon d'entraînement et d'améliorer la stabilité de la prédiction.

Ce modèle présente plusieurs avantages :

— il gère bien les relations non linéaires ;

— il est robuste aux valeurs aberrantes ;

— il nécessite peu de prétraitement ;

— il donne généralement de bonnes performances sur des données tabulaires.

Dans notre application, le Random Forest est utilisé comme modèle principal pour l'approximation du prix des options.

3.8 Entraînement du modèle

L'entraînement du Random Forest consiste à construire plusieurs arbres de décision à partir de la base d'apprentissage. Chaque arbre reçoit un échantillon bootstrap des données et apprend une série de règles de décision permettant de relier les variables d'entrée au prix cible.

Pour chaque observation de l'ensemble d'entraînement, on dispose d'un vecteur de paramètres financiers x_i et d'un prix cible y_i calculé par Black–Scholes :

$$(x_i, y_i), \qquad y_i = V_{BS}(x_i).$$

Chaque arbre cherche à partitionner l'espace des variables de manière à réduire l'erreur de prédiction à l'intérieur des feuilles. Une fois les arbres construits, la prédiction finale est obtenue par moyenne :

$$\widehat{y}_{RF} = \frac{1}{B}\sum_{b=1}^{B} T_b(x),$$

où $T_b(x)$ désigne la prédiction du b-ième arbre.

Dans l'application, le modèle entraîné est ensuite sauvegardé afin de pouvoir prédire rapidement le prix d'une nouvelle option sans devoir relancer toute la phase d'apprentissage.

3.9 Évaluation du modèle

L'évaluation du modèle est réalisée sur l'ensemble de test. Les métriques utilisées dans ce projet sont la MAE, la RMSE et le coefficient de détermination R^2.

3.9.1 Erreur absolue moyenne

La MAE mesure l'erreur absolue moyenne entre les valeurs réelles et les valeurs prédites :

$$MAE = \frac{1}{n}\sum_{i=1}^{n} |y_i - \widehat{y}_i|.$$

La MAE s'exprime dans la même unité que le prix de l'option. Elle donne donc une interprétation directe de l'erreur moyenne commise par le modèle. Une MAE faible indique que les prédictions sont proches des prix cibles.

3.9.2 Erreur quadratique moyenne

La RMSE est définie par :

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \widehat{y}_i)^2}.$$

La RMSE est également exprimée dans l'unité du prix. Elle est plus sensible aux grandes erreurs que la MAE, ce qui permet de détecter les cas où le modèle se trompe fortement. Elle pénalise davantage les grandes erreurs, car les écarts sont élevés au carré.

3.9.3 Coefficient de détermination

Le coefficient de détermination R^2 mesure la proportion de variance expliquée par le modèle :

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \widehat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \overline{y})^2}.$$

Contrairement à la MAE et à la RMSE, le coefficient R^2 est sans unité. Il mesure la capacité globale du modèle à expliquer les variations du prix dans l'ensemble de test. Un R^2 proche de 1 signifie que le modèle explique très bien la variabilité du prix.

3.10 Prédiction du prix d'une option européenne

Une fois le modèle entraîné, l'utilisateur peut saisir les paramètres d'une option européenne. Le modèle transforme ces paramètres en variables explicatives, applique les transformations nécessaires, puis retourne le prix prédit :

$$\widehat{V}_{ML} = \widehat{f}(S_0, K, r, \sigma, T, \text{type}).$$

L'intérêt principal de cette approche est la rapidité de prédiction. Après entraînement, le calcul est généralement quasi instantané.

Dans l'interface développée, la partie Machine Learning suit une logique en plusieurs étapes : génération d'une base de données de prix Black–Scholes, entraînement du modèle Random Forest, évaluation par les métriques MAE, RMSE et R^2, puis prédiction du prix d'une nouvelle option. Cette organisation permet de visualiser clairement le passage entre la théorie financière, les données simulées et l'approximation statistique.

3.11 Intérêt et limites de l'approche Machine Learning

Le Machine Learning présente plusieurs avantages :

— prédiction rapide après entraînement ;
— capacité à approximer des fonctions non linéaires ;
— possibilité d'être utilisé lorsque la formule analytique est indisponible ;
— intégration facile dans une application interactive.

Cependant, il présente aussi des limites :

— il dépend fortement de la qualité des données d'entraînement ;
— il ne garantit pas l'absence d'arbitrage ;
— il peut mal extrapoler hors du domaine appris ;
— il est moins interprétable qu'une formule analytique ;
— il nécessite une étape d'entraînement préalable.

3.12 Objectif de la comparaison numérique

La comparaison numérique a pour objectif d'évaluer les différentes méthodes selon plusieurs critères :

— le prix obtenu ;

— l'erreur absolue par rapport à Black–Scholes ;

— l'erreur relative ;

— le temps de calcul ;

— la stabilité numérique ;

— la flexibilité d'utilisation.

Black–Scholes est utilisé comme référence car il fournit un prix analytique exact dans le cadre des hypothèses du modèle.

3.13 Paramètres numériques utilisés

Les paramètres de référence utilisés dans la comparaison sont les suivants :

Paramètre	**Valeur**	**Interprétation**
S_0	100	Prix initial du sous-jacent.
K	100	Option à la monnaie.
r	5%	Taux sans risque annualisé.
σ	20%	Volatilité annualisée.
T	1 an	Maturité.
M	10 000	Nombre de simulations Monte–Carlo.
N	252	Nombre de pas Euler–Maruyama.

Table 3.3 – Paramètres numériques utilisés pour la comparaison des méthodes.

3.14 Comparaison pour une option Call européenne

Pour une option Call européenne à la monnaie avec les paramètres précédents, le prix Black–Scholes est proche de :

$$C_{BS} \approx 10.4506.$$

Monte–Carlo et Euler–Maruyama doivent converger vers cette valeur lorsque le nombre de simulations et le nombre de pas augmentent. Le modèle Machine Learning doit également produire une valeur proche si la base d'entraînement couvre correctement cette zone de paramètres.

3.15 Comparaison pour une option Put européenne

Pour un Put européen, le prix peut être contrôlé à l'aide de la parité Call–Put :

$$P = C - S_0 + Ke^{-rT}.$$

Cette relation est utile pour vérifier la cohérence des calculs, notamment dans l'application.

3.16 Comparaison des prix obtenus

Méthode	**Prix obtenu**	**Nature de l'erreur**
Black–Scholes	Prix exact de référence	Erreur numérique négligeable.
Monte–Carlo	Proche de Black–Scholes	Erreur statistique.
Euler–Maruyama	Proche de Black–Scholes	Erreur statistique et biais de discrétisation.
Machine Learning	Dépend du modèle entraîné	Erreur d'approximation.

Table 3.4 – Comparaison générale des prix obtenus par les différentes méthodes.

3.17 Comparaison des erreurs par rapport à Black–Scholes

L'erreur absolue est définie par :

$$E_{abs} = |\widehat{V} - V_{BS}|.$$

L'erreur relative est donnée par :

$$E_{rel} = \frac{|\widehat{V} - V_{BS}|}{V_{BS}} \times 100.$$

Ces deux mesures permettent de comparer les méthodes sur une même base.

3.18 Comparaison des temps de calcul

Le temps de calcul varie fortement d'une méthode à l'autre. Black–Scholes est très rapide car il repose sur une formule fermée. Monte–Carlo est plus lent car il nécessite un grand nombre de simulations. Euler–Maruyama est encore plus coûteux car il simule toutes les trajectoires pas à pas. Le Random Forest est rapide en prédiction par rapport aux simulations répétées, car il applique simplement les arbres déjà entraînés. Son coût dépend principalement du nombre d'arbres et de leur profondeur, tandis que le coût le plus important reste la phase d'entraînement.

Méthode	**Complexité**	**Interprétation**
Black–Scholes	$O(1)$	Calcul direct par formule analytique.
Monte–Carlo	$O(M)$	Dépend du nombre de simulations.
Euler–Maruyama	$O(MN)$	Dépend du nombre de simulations et du nombre de pas.
Machine Learning	$O(Bd)$ en prédiction	Dépend du nombre d'arbres B et de leur profondeur d, mais reste rapide après entraînement.

Table 3.5 – Comparaison des complexités de calcul.

3.19 Analyse de la convergence Monte–Carlo

La convergence de Monte–Carlo suit :

$$E_{MC} = O\left(\frac{1}{\sqrt{M}}\right).$$

Cette convergence est relativement lente. Pour améliorer la précision d'un facteur 10, il faut multiplier le nombre de simulations par 100. Dans l'application, une courbe de convergence

permet de visualiser le rapprochement progressif de l'estimateur Monte–Carlo vers le prix Black–Scholes.

3.20 Analyse de l'effet du pas de discrétisation dans Euler–Maruyama

L'erreur de discrétisation d'Euler–Maruyama diminue lorsque N augmente. Comme le pas de temps est donné par $\Delta t = T/N$, une diminution de Δt améliore généralement l'approximation de la trajectoire.

Dans le cadre de l'étude numérique, on observe le biais par rapport au prix de référence Black–Scholes :

$$E_{EM} = |\widehat{V}_{EM} - V_{BS}|.$$

Ce biais tend généralement à diminuer lorsque N augmente, même si l'estimation reste également affectée par l'erreur statistique liée au nombre de simulations M.

Le choix de N représente donc un compromis entre précision et temps de calcul. Une valeur faible de N donne un calcul rapide mais moins précis, tandis qu'une valeur élevée réduit le biais mais augmente le coût computationnel.

3.21 Analyse des performances du modèle Machine Learning

Les performances du modèle Machine Learning sont évaluées à l'aide de la MAE, de la RMSE et du R^2. Un bon modèle doit présenter :

- une MAE faible ;
- une RMSE faible ;
- un R^2 proche de 1.

Cependant, même avec de bonnes métriques, le modèle doit être utilisé avec prudence. Une bonne performance sur l'ensemble de test ne garantit pas une bonne extrapolation en dehors des plages de paramètres utilisées pour l'entraînement.

3.22 Synthèse comparative des méthodes

Critère	**Black–Scholes**	**Monte–Carlo**	**Euler–Maruyama**	**Machine Learning**
Précision	Exacte dans le cadre du modèle	Bonne si M est grand	Bonne si M et N sont grands	Dépend de l'entraînement
Erreur principale	Aucune dans le cadre du modèle	Erreur statistique	Erreur statistique et biais de discrétisation	Erreur d'approximation
Vitesse	Très rapide	Moyenne	Plus lente	Très rapide après entraînement
Flexibilité	Limitée	Forte	Très forte	Moyenne
Interprétabilité	Très forte	Bonne	Bonne	Plus faible
Usage	Référence analytique	Simulation	EDS complexes	Approximation rapide

Table 3.6 – Synthèse comparative des méthodes de pricing.

3.23 Conclusion du chapitre

Ce chapitre a présenté l'utilisation du Machine Learning comme méthode d'approximation appliquée au pricing des options européennes. Dans ce projet, le modèle Random Forest ne remplace pas la théorie financière : il apprend à reproduire les prix générés par Black–Scholes à partir des paramètres de l'option.

Le Random Forest repose sur une logique d'ensemble : plusieurs arbres de décision sont entraînés sur des échantillons bootstrap, puis leurs prédictions sont moyennées afin de stabiliser le résultat. Ses performances sont évaluées à l'aide de la MAE, de la RMSE et du coefficient de détermination R^2.

La comparaison numérique montre que les méthodes étudiées sont complémentaires : Black–Scholes fournit la référence analytique, Monte–Carlo approxime l'espérance par simulation, Euler–Maruyama discrétise les trajectoires, et le Random Forest fournit une prédiction rapide après apprentissage.

4 Présentation de l'application développée

4.1 Introduction

Dans le cadre de ce projet, une application web interactive a été développée afin de mettre en pratique les méthodes théoriques et numériques présentées dans les chapitres précédents. Cette application, intitulée *Option Pricing Lab*, permet de calculer, comparer et visualiser les prix d'options européennes à l'aide de plusieurs approches : Black–Scholes, Monte–Carlo, Euler–Maruyama et Machine Learning.

L'objectif de ce chapitre est de présenter l'application développée, son architecture générale, les technologies utilisées, les interfaces principales ainsi que les fonctionnalités d'export et de visualisation. L'application constitue une partie importante du projet, car elle permet de transformer les concepts mathématiques en résultats numériques exploitables.

4.2 Objectif de l'application

L'application a été conçue pour répondre à plusieurs objectifs :

— calculer le prix d'une option européenne par différentes méthodes ;
— comparer les prix obtenus avec la référence Black–Scholes ;
— visualiser les trajectoires simulées du sous-jacent ;
— analyser la convergence de Monte–Carlo ;
— étudier l'effet du nombre de pas dans Euler–Maruyama ;
— entraîner un modèle de Machine Learning pour approximer le prix ;
— afficher les erreurs et les temps de calcul ;
— exporter les résultats en CSV, Excel ou PDF.

L'application joue donc un rôle à la fois pédagogique et expérimental. Elle permet à l'utilisateur de modifier les paramètres financiers et d'observer immédiatement leur impact sur les résultats.

L'intérêt de l'application ne réside pas seulement dans le calcul automatique des prix. Elle permet surtout de visualiser le lien entre les formules théoriques et les résultats numériques.

Par exemple, Black–Scholes fournit une valeur de référence, Monte–Carlo montre comment une moyenne simulée converge vers cette valeur, Euler–Maruyama illustre la discrétisation d'une EDS, et le Random Forest montre comment une fonction de pricing peut être approximée après apprentissage.

4.3 Technologies utilisées

L'application repose sur une architecture web développée principalement en Python. Les principales technologies utilisées sont les suivantes :

Technologie	**Rôle dans l'application**
Python	Langage principal utilisé pour les calculs financiers, les simulations et le Machine Learning.
Flask	Framework web utilisé pour gérer les routes, les pages et les interactions entre l'utilisateur et les modules de calcul.
NumPy	Bibliothèque utilisée pour les calculs vectorisés et la génération des variables aléatoires.
SciPy	Bibliothèque utilisée pour les fonctions statistiques, notamment la loi normale.
pandas	Manipulation des données et préparation des exports.
scikit-learn	Entraînement du modèle Random Forest utilisé comme *surrogate model*.
Chart.js	Visualisation des résultats sous forme de graphiques interactifs.
HTML/CSS	Construction et mise en forme des interfaces.
JavaScript	Interactions dynamiques avec l'utilisateur et mise à jour des résultats.

Table 4.1 – Technologies utilisées dans l'application Option Pricing Lab.

4.4 Architecture générale de l'application

L'application est organisée selon une architecture modulaire. Cette organisation permet de séparer clairement la logique de calcul financier, les traitements liés au Machine Learning, les services intermédiaires et l'interface utilisateur. Une telle structure rend le projet plus lisible, plus facile à maintenir et plus simple à faire évoluer.

Le principe général consiste à isoler les fonctions de calcul dans des modules indépendants de Flask. Ainsi, les méthodes de pricing peuvent être testées séparément, sans dépendre directement de l'interface web. Les routes Flask jouent ensuite le rôle d'intermédiaire entre l'utilisateur et ces modules de calcul.

Couche	**Rôle dans l'application**
Couche de calcul financier	Elle contient les fonctions principales de pricing : formule de Black–Scholes, simulation Monte–Carlo, simulation Euler–Maruyama, calcul des erreurs et métriques comparatives. Cette couche regroupe la partie scientifique du projet et reste indépendante de l'interface web.
Couche Machine Learning	Elle regroupe les traitements liés à l'apprentissage automatique : génération de la base d'apprentissage, transformation des variables, séparation train/test, entraînement du modèle Random Forest, évaluation par MAE, RMSE et R^2, puis prédiction du prix d'une nouvelle option.
Couche services	Elle assure le lien entre les routes Flask et les fonctions de calcul. Elle prépare les données reçues depuis l'interface, appelle la méthode appropriée, formate les résultats et les transmet aux pages HTML ou aux réponses JSON.
Couche interface utilisateur	Elle contient les fichiers HTML, CSS et JavaScript. Elle permet à l'utilisateur de saisir les paramètres, lancer les calculs, visualiser les résultats, comparer les méthodes et exporter les données sous différents formats.

Table 4.2 – Organisation modulaire de l'application développée.

Cette architecture permet donc de distinguer la partie scientifique de l'application, qui regroupe les modèles de pricing et les algorithmes numériques, de la partie web, qui assure

l'interaction avec l'utilisateur. Elle facilite également l'ajout futur de nouvelles méthodes, par exemple une autre technique de simulation, un modèle de volatilité stochastique ou un nouveau modèle de Machine Learning.

4.5 Structure générale du projet

La structure générale de l'application est organisée comme suit :

```
option_pricing/
├── app.py
├── config.py
├── core/
│   ├── black_scholes.py
│   ├── monte_carlo.py
│   ├── euler_maruyama.py
│   └── comparison.py
├── ml/
│   ├── dataset_generator.py
│   ├── train_model.py
│   ├── predict_model.py
│   └── preprocessing.py
├── services/
│   ├── pricing_service.py
│   ├── simulation_service.py
│   ├── ml_service.py
│   └── export_service.py
├── templates/
│   ├── base.html
│   ├── index.html
│   ├── black_scholes.html
│   ├── monte_carlo.html
│   ├── euler_maruyama.html
│   ├── machine_learning.html
│   └── comparaison.html
└── static/
    ├── css/
    └── js/
```

Cette organisation rend le projet plus clair et facilite la maintenance.

4.6 Tableau de bord principal

Le tableau de bord principal présente une vue générale de l'application. Il permet de saisir les paramètres de base :

$$S_0, \quad K, \quad r, \quad \sigma, \quad T, \quad M, \quad N.$$

L'utilisateur peut ensuite lancer le calcul et obtenir les prix estimés par les différentes méthodes.

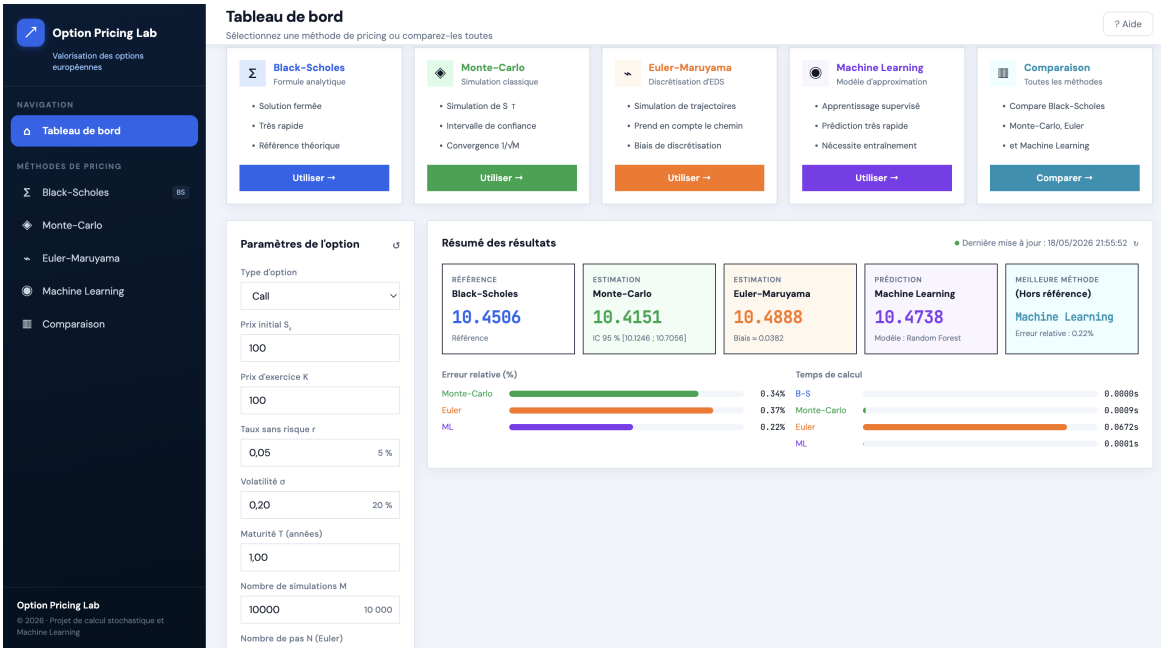


Figure 4.1 – Tableau de bord principal de l'application Option Pricing Lab.

4.7 Interface de calcul Black–Scholes

La page Black–Scholes permet de calculer le prix analytique d'un Call ou d'un Put européen à partir de la formule de Black–Scholes, avec la possibilité d'introduire un taux de dividende continu q. Elle constitue la référence théorique de l'application, car les autres méthodes sont comparées au prix obtenu par cette formule.

Les informations affichées dans cette interface sont principalement :

— les paramètres financiers de l'option : S_0, K, r, σ, T et éventuellement q ;

— le type de l'option : Call ou Put ;

— le prix analytique obtenu ;

— le payoff à maturité ;

— l'évolution des prix Call et Put en fonction du sous-jacent ;

— les hypothèses principales du modèle.

Cette interface permet de comprendre rapidement l'effet des paramètres financiers sur le prix analytique.

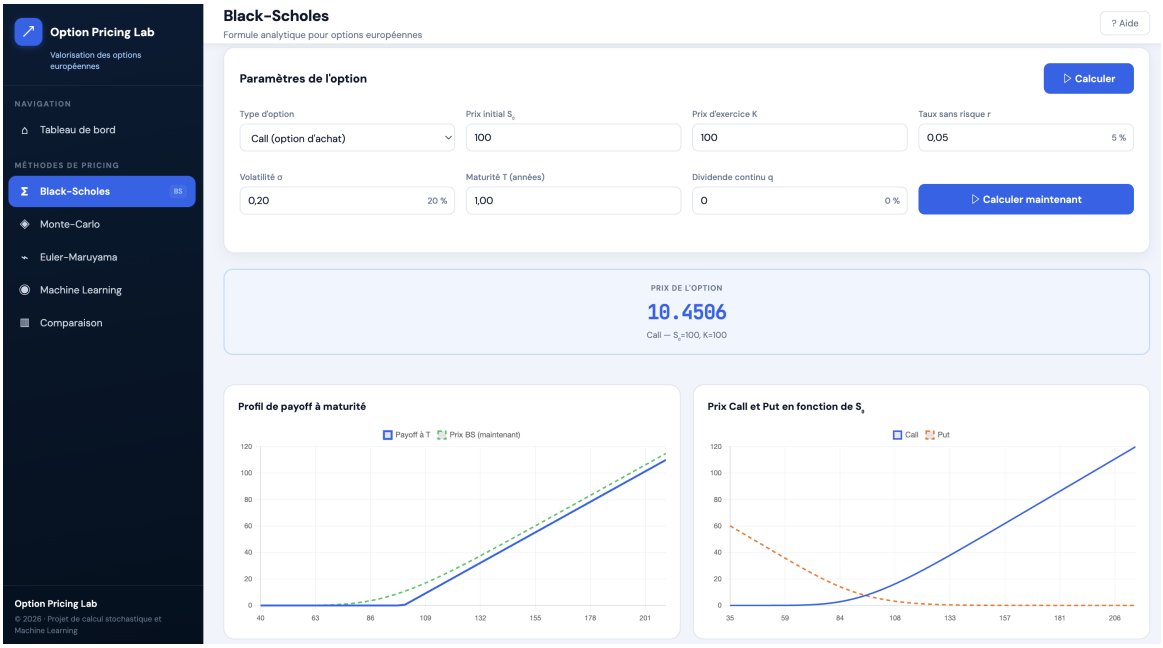


Figure 4.2 – Interface de calcul Black–Scholes.

La figure montre que l'interface Black–Scholes regroupe les paramètres de l'option, la formule utilisée, le prix analytique obtenu et les graphiques associés au payoff et à l'évolution du prix de l'option. Cette organisation permet à l'utilisateur de relier directement les paramètres financiers saisis au résultat théorique du modèle.

4.8 Interface de simulation Monte–Carlo

La page Monte–Carlo permet de simuler un grand nombre de scénarios du prix terminal S_T. L'utilisateur peut choisir le nombre de simulations M et observer l'effet de ce nombre sur la précision du résultat.

Cette interface affiche généralement :

— le prix estimé par Monte–Carlo ;

— le prix de référence Black–Scholes ;

— l'erreur absolue et l'erreur relative ;

— l'intervalle de confiance à 95% ;

— le temps de calcul ;

— une comparaison synthétique avec le prix Black–Scholes.

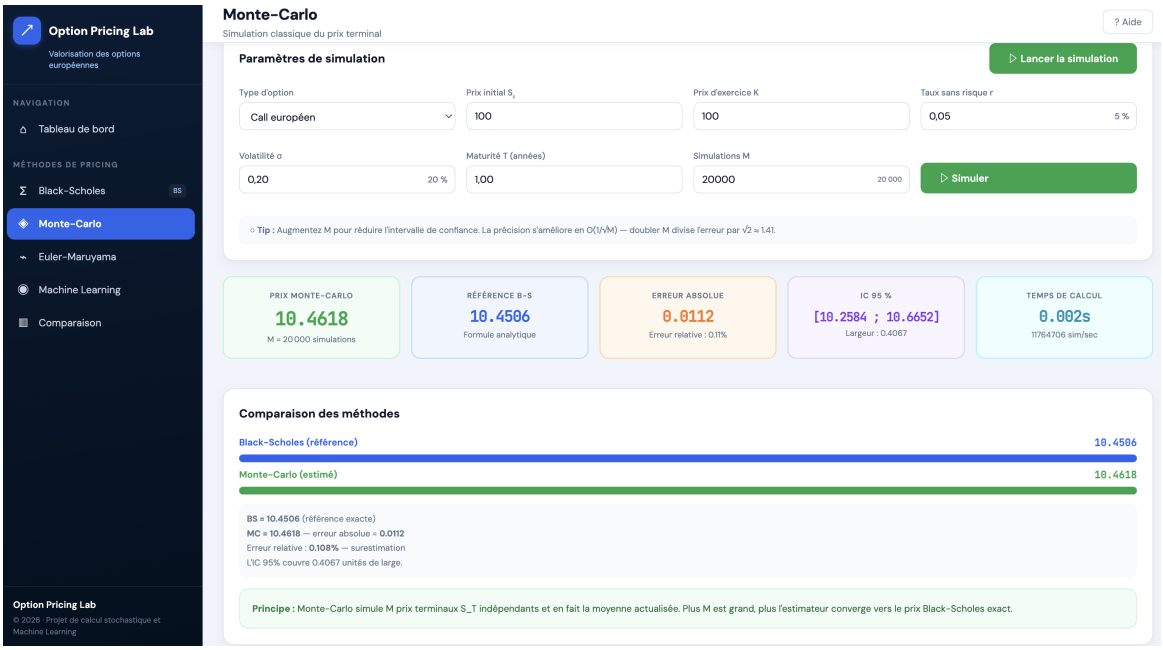


Figure 4.3 – Interface de simulation Monte–Carlo.

La figure met en évidence le caractère statistique de la méthode Monte–Carlo. Contrairement à Black–Scholes, le prix obtenu dépend du nombre de simulations et de la graine aléatoire utilisée. L'affichage de l'intervalle de confiance permet de mesurer l'incertitude associée à l'estimation numérique.

4.8.1 Graphe de convergence Monte–Carlo

Le graphe de convergence montre comment l'estimateur Monte–Carlo se rapproche progressivement du prix Black–Scholes lorsque le nombre de simulations augmente.

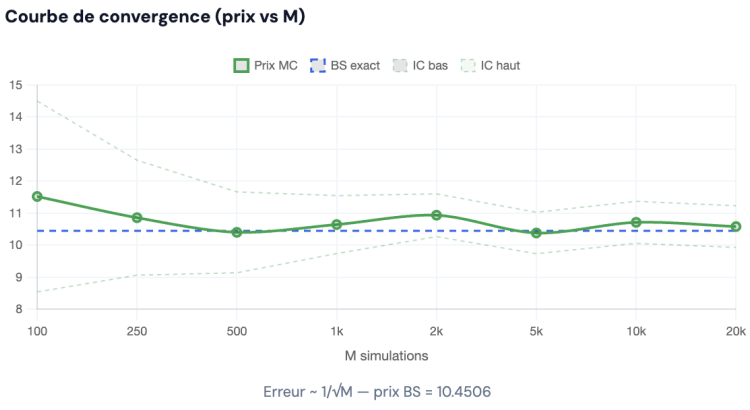


Figure 4.4 – Courbe de convergence de la méthode Monte–Carlo.

La figure 4.4 illustre la convergence progressive de l'estimateur Monte–Carlo vers le prix de référence obtenu par la formule de Black–Scholes. Lorsque le nombre de simulations est faible, par exemple pour $M = 100$ ou $M = 250$, l'estimation est plus instable et peut s'écarter

sensiblement de la valeur théorique. À mesure que M augmente, la courbe du prix Monte–Carlo se rapproche de la ligne de référence Black–Scholes.

Les deux bornes de l'intervalle de confiance montrent également que l'incertitude statistique diminue lorsque le nombre de simulations augmente. Cette observation confirme le résultat théorique selon lequel l'erreur de Monte–Carlo décroît en ordre $1/\sqrt{M}$. Ainsi, la figure met en évidence l'intérêt d'augmenter le nombre de simulations pour obtenir une estimation plus stable, tout en rappelant que cette amélioration se fait au prix d'un temps de calcul plus élevé.

4.9 Interface Euler–Maruyama

La page Euler–Maruyama permet de simuler des trajectoires complètes du sous-jacent. Contrairement à Monte–Carlo direct, qui simule seulement S_T, Euler–Maruyama discrétise toute l'évolution du processus entre 0 et T.

Cette interface permet notamment de visualiser :

— les trajectoires simulées du sous-jacent ;

— le prix estimé par Euler–Maruyama ;

— le prix de référence Black–Scholes ;

— le biais absolu et relatif par rapport à Black–Scholes ;

— le pas de temps $\Delta t = T/N$;

— le temps de calcul ;

— la comparaison avec Black–Scholes et Monte–Carlo.

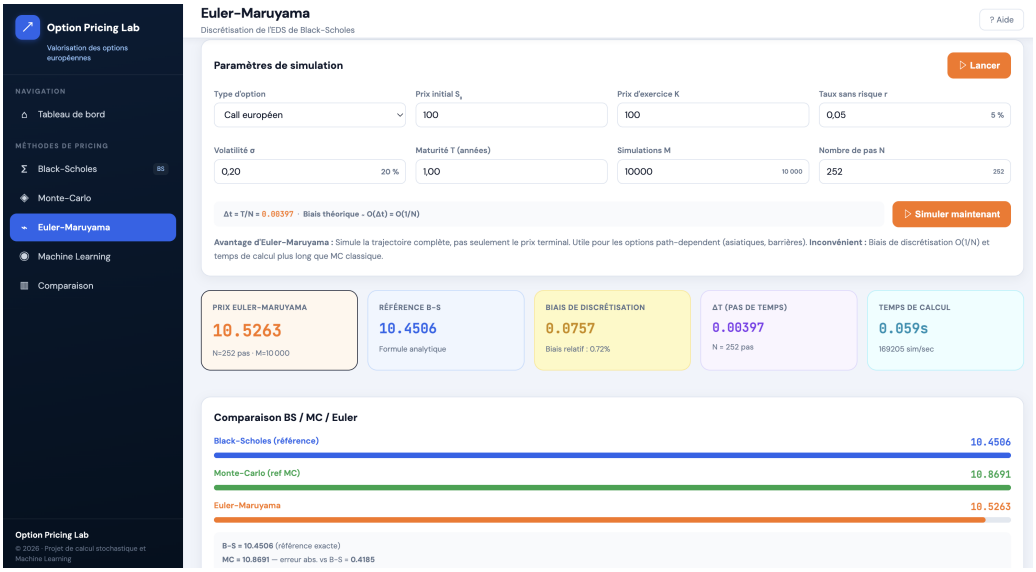


Figure 4.5 – Interface Euler–Maruyama.

Cette interface illustre la différence entre une simulation directe du prix terminal et une simulation trajectorielle. Le schéma d'Euler–Maruyama permet de suivre l'évolution du sous-jacent sur toute la grille temporelle, ce qui rend la méthode plus générale que Monte–Carlo direct, mais aussi plus coûteuse en temps de calcul.

4.9.1 Trajectoires simulées

Les trajectoires simulées permettent d'observer visuellement l'effet de la volatilité sur l'évolution du sous-jacent. Elles illustrent la nature aléatoire du mouvement brownien géométrique.

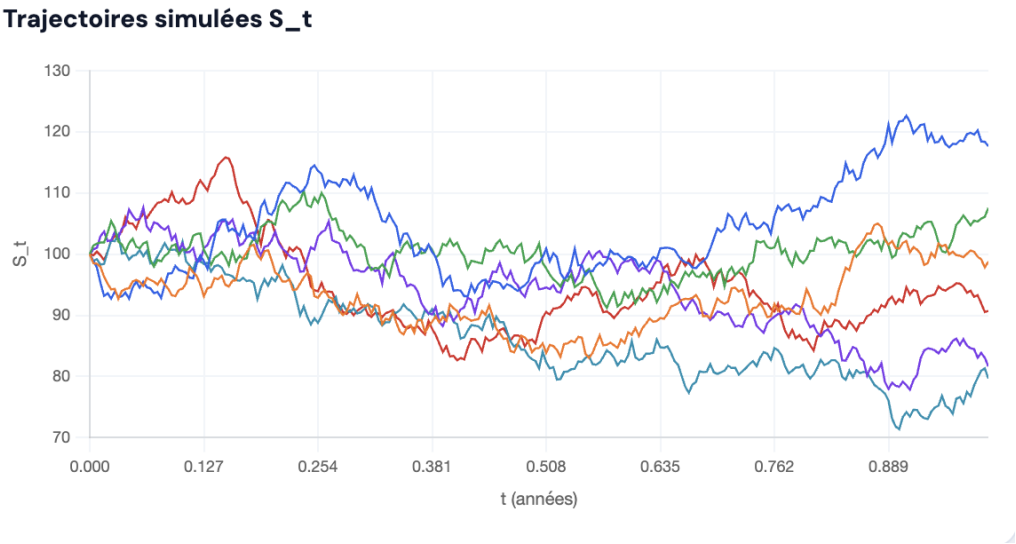


Figure 4.6 – Trajectoires simulées du sous-jacent par Euler–Maruyama.

4.10 Interface Machine Learning

La page Machine Learning permet d'entraîner un modèle de régression sur une base de prix générés par Black–Scholes. Dans cette version du projet, le modèle utilisé est le Random Forest. L'utilisateur peut lancer le pipeline d'apprentissage, générer une base de données de prix Black–Scholes, entraîner le modèle, consulter les métriques d'évaluation et obtenir une prédiction pour une nouvelle option.

L'interface affiche :

— le modèle Random Forest utilisé ;

— le nombre d'observations générées ;

— les métriques MAE, RMSE et R^2 ;

— le prix prédit par le modèle ;

— la comparaison avec le prix Black–Scholes.

Cette page permet d'illustrer le rôle du Machine Learning comme méthode d'approximation de la fonction de pricing.

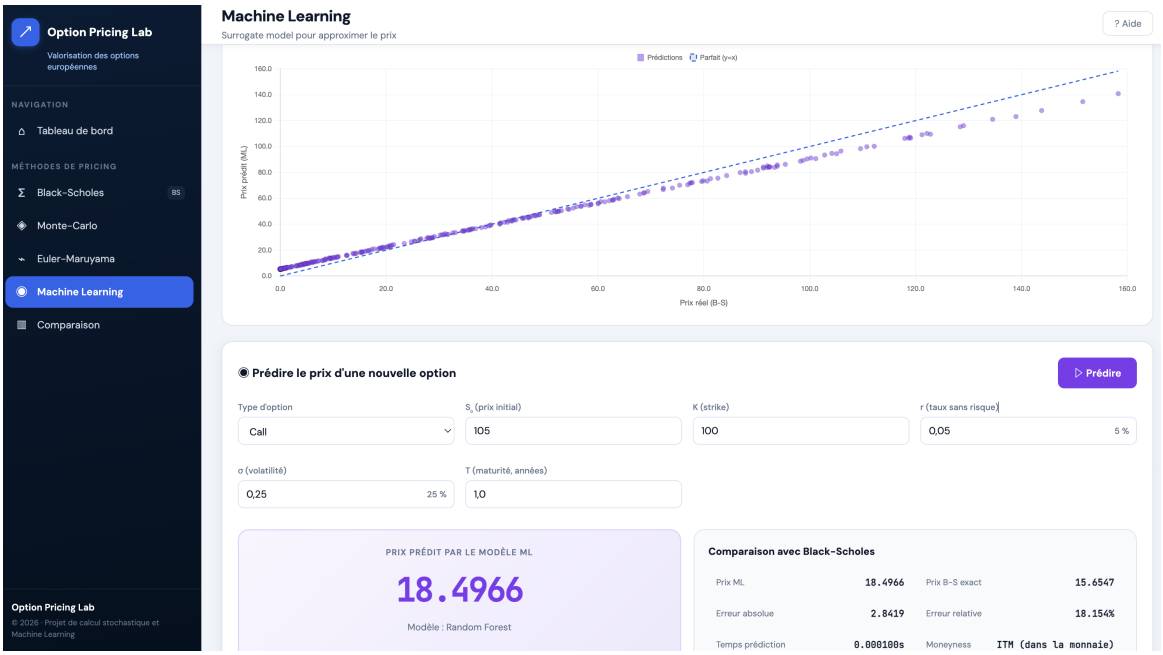


Figure 4.7 – Interface Machine Learning pour l'entraînement et la prédiction.

4.11 Tableau comparatif et visualisations graphiques

La page de comparaison constitue l'une des parties les plus importantes de l'application. Elle rassemble les résultats des différentes méthodes dans un même espace afin de faciliter l'analyse et l'interprétation des écarts entre les approches étudiées.

Le tableau comparatif présente, pour chaque méthode de pricing, les principaux résultats numériques : le prix obtenu, l'erreur absolue, l'erreur relative par rapport à la référence Black–Scholes, le temps de calcul ainsi que le statut de la méthode. Cette organisation permet de comparer rapidement les performances de Black–Scholes, Monte–Carlo, Euler–Maruyama et Machine Learning selon plusieurs critères.

Élément affiché	**Rôle dans l'analyse comparative**
Méthode utilisée	Identifie l'approche de pricing appliquée : Black–Scholes, Monte–Carlo, Euler–Maruyama ou Machine Learning.
Prix obtenu	Permet de comparer directement les valeurs calculées par les différentes méthodes.
Erreur absolue	Mesure l'écart numérique entre le prix estimé et le prix de référence Black–Scholes.
Erreur relative	Exprime l'erreur en pourcentage, ce qui facilite l'interprétation de la précision.
Temps de calcul	Évalue le coût computationnel de chaque méthode.
Statut	Indique la position ou la qualité de la méthode dans la comparaison globale.

Table 4.3 – Informations affichées dans le tableau comparatif des méthodes.

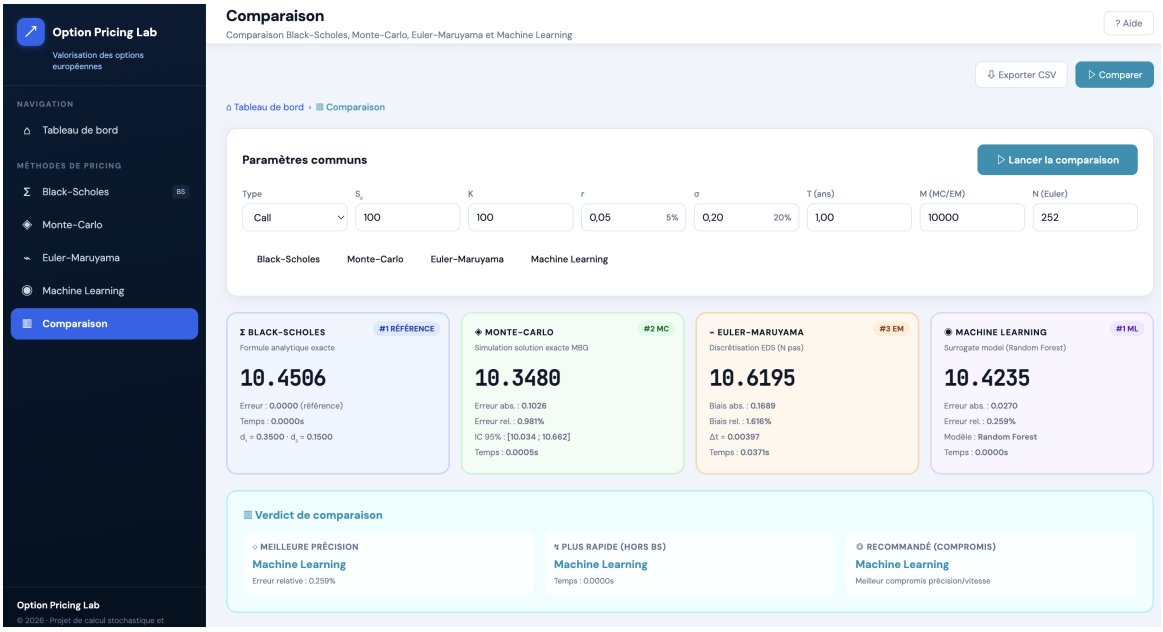


Figure 4.8 – Interface de comparaison des méthodes de pricing.

La figure 4.8 montre que l'application ne se limite pas à fournir un prix isolé. Elle organise les résultats de manière comparative, ce qui permet d'identifier rapidement les écarts entre les méthodes. Black–Scholes joue le rôle de référence analytique, tandis que Monte–Carlo, Euler–Maruyama et le modèle Random Forest sont évalués selon leur proximité avec cette référence et leur temps de calcul.

4.12 Export des résultats

L'application permet également d'exporter les résultats obtenus afin de les exploiter en dehors de l'interface web. Trois formats sont proposés : CSV, Excel et PDF. Le format CSV permet d'obtenir les données brutes, le format Excel fournit un tableau formaté plus lisible, tandis que le PDF permet de générer un rapport complet du scénario étudié.

Format	**Utilité principale**
CSV	Export simple des résultats sous forme de données brutes, exploitable dans Excel, Google Sheets, Python ou R.
Excel	Export structuré sous forme de tableau formaté, adapté à une analyse plus professionnelle des paramètres, prix, erreurs et temps de calcul.
PDF	Génération d'un rapport complet destiné à la présentation, à l'archivage ou à l'intégration dans un document académique.

Table 4.4 – Formats d'export proposés par l'application.

4.13 Apports de l'application

L'application apporte plusieurs bénéfices :

— elle relie la théorie mathématique à une implémentation concrète ;
— elle permet de comparer plusieurs méthodes dans les mêmes conditions ;
— elle facilite la compréhension de la convergence Monte–Carlo ;
— elle montre l'effet du pas de discrétisation dans Euler–Maruyama ;
— elle illustre le rôle du Machine Learning comme méthode d'approximation ;
— elle permet d'exporter les résultats pour une exploitation académique.

Elle constitue donc un support pédagogique complet pour comprendre le pricing des options européennes.

4.14 Limites de l'application

Même si l'application permet de comparer plusieurs méthodes, elle reste limitée au cadre des options européennes standards. Les options américaines, asiatiques, barrières ou exotiques ne sont pas directement prises en charge dans cette version. De plus, le modèle de Black–Scholes utilisé repose sur des hypothèses simplificatrices : volatilité constante, taux sans

risque constant, absence de coûts de transaction et absence de sauts de marché. L'interface permet d'introduire un dividende continu q, mais elle ne traite pas des modèles plus avancés comme la volatilité stochastique ou les processus à sauts.

L'approche Machine Learning dépend également de la base d'apprentissage. Si les paramètres saisis par l'utilisateur sont très éloignés des plages utilisées pour entraîner le modèle, la prédiction peut être moins fiable.

4.15 Perspectives d'amélioration

Plusieurs améliorations peuvent être envisagées :

— ajouter le pricing d'options américaines ;
— intégrer la méthode de Longstaff–Schwartz ;
— ajouter des options asiatiques et barrières ;
— intégrer des modèles à volatilité stochastique ;
— tester d'autres modèles de Machine Learning, comme le MLP, uniquement en extension comparative ;
— sauvegarder les scénarios dans une base de données ;
— enrichir les exports PDF avec les graphiques générés.

Ces perspectives permettraient de rendre l'application plus complète et plus proche des outils utilisés en finance quantitative professionnelle.

4.16 Conclusion du chapitre

Ce chapitre a présenté l'application développée dans le cadre du projet. L'application *Option Pricing Lab* permet de calculer, comparer et visualiser les prix d'options européennes par différentes méthodes. Elle intègre Black–Scholes, Monte–Carlo, Euler–Maruyama et un module Machine Learning basé sur Random Forest dans une interface unique.

Grâce aux visualisations, aux tableaux comparatifs et aux fonctionnalités d'export, l'application permet de passer de la théorie à la pratique. Elle constitue ainsi un outil pédagogique et expérimental permettant de mieux comprendre les forces, les limites et les complémentarités des méthodes de pricing étudiées.

Conclusion générale

Ce travail a permis d'étudier la valorisation des options européennes à travers une progression allant du modèle analytique vers les méthodes numériques, puis vers l'approximation par Machine Learning. Le modèle de Black–Scholes constitue la référence théorique du projet, car il fournit une formule fermée rapide dans le cadre de ses hypothèses. La méthode de Monte–Carlo permet ensuite de retrouver ce prix par simulation de scénarios, en approchant l'espérance actualisée du payoff. Le schéma d'Euler–Maruyama introduit une approche trajectorielle fondée sur la discrétisation de l'équation différentielle stochastique du sous-jacent. Enfin, le modèle Random Forest est utilisé comme *surrogate model*, c'est-à-dire comme une approximation statistique de la fonction de pricing générée par Black–Scholes.

L'étude comparative montre que ces méthodes ne répondent pas exactement au même objectif. Black–Scholes est la méthode la plus rapide et la plus interprétable lorsque ses hypothèses sont vérifiées. Monte–Carlo est plus flexible, mais son erreur statistique décroît lentement avec le nombre de simulations. Euler–Maruyama est plus général, car il permet de simuler des trajectoires complètes, mais il introduit un biais de discrétisation dépendant du pas de temps. Le Random Forest, quant à lui, permet une prédiction rapide après entraînement, mais sa fiabilité dépend de la qualité et de la couverture de la base d'apprentissage.

L'application développée, *Option Pricing Lab*, permet de mettre en pratique l'ensemble de ces méthodes dans une interface unique. Elle facilite la comparaison des prix, l'analyse des erreurs, l'observation de la convergence Monte–Carlo, l'étude de la discrétisation Euler–Maruyama et l'export des résultats. Elle constitue ainsi un support pédagogique et expérimental permettant de relier la finance quantitative, le calcul stochastique, les méthodes numériques et l'apprentissage automatique.

En conclusion, ce projet montre que les différentes approches étudiées sont complémentaires. La théorie fournit le cadre de référence, la simulation permet d'approcher les espérances, la discrétisation permet de traiter les dynamiques stochastiques trajectorielles, et le Machine Learning offre une approximation rapide après apprentissage. Cette complémentarité représente l'un des intérêts majeurs de la finance quantitative moderne.

Bibliographie

[1] Black, F. and Scholes, M. (1973). *The Pricing of Options and Corporate Liabilities*. Journal of Political Economy, 81(3), 637–654.

[2] Merton, R. C. (1973). *Theory of Rational Option Pricing*. Bell Journal of Economics and Management Science, 4(1), 141–183.

[3] Boyle, P. P. (1977). *Options : A Monte Carlo Approach*. Journal of Financial Economics, 4(3), 323–338.

[4] Kloeden, P. E. and Platen, E. (1992). *Numerical Solution of Stochastic Differential Equations*. Springer.

[5] Glasserman, P. (2004). *Monte Carlo Methods in Financial Engineering*. Springer.

[6] Hull, J. C. (2018). *Options, Futures, and Other Derivatives*. Pearson.

[7] Breiman, L. (2001). *Random Forests*. Machine Learning, 45, 5–32.

[8] Itô, K. (1944). *Stochastic Integral*. Proceedings of the Imperial Academy, 20(8), 519–524. DOI : 10.3792/pia/1195572786.

[9] Øksendal, B. (2003). *Stochastic Differential Equations : An Introduction with Applications*. 6th Edition, Springer, Berlin.

[10] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). *Scikit-learn : Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825–2830.

[11] Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., del Río, J. F., Wiebe, M., Peterson, P., Gérard-Marchant, P., Sheppard, K., Reddy, T., Weckesser, W., Abbasi, H., Gohlke, C., & Oliphant, T. E. (2020). *Array programming with NumPy*. Nature, 585, 357–362. DOI : 10.1038/s41586-020-2649-2.